

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A PROJECT REPORT

ON

A PRIVACY-PRESERVING EDGE TINYML SYSTEM FOR ACOUSTIC COMFORT ESTIMATION USING HYBRID DSP–CNN FUSION AND VISUAL FEEDBACK

Submitted By:

Plan Ghimire	(080BEI028)
Pranjal Shrestha	(080BEI033)
Sagun Acharya	(080BEI037)
Samman Ghale	(080BEI039)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

August, 2026



TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A PROJECT REPORT

ON

A PRIVACY-PRESERVING EDGE TINYML SYSTEM FOR ACOUSTIC COMFORT ESTIMATION USING HYBRID DSP–CNN FUSION AND VISUAL FEEDBACK

Submitted By:

Plan Ghimire (080BEI028)

Pranjal Shrestha (080BEI033)

Sagun Acharya (080BEI037)

Samman Ghale (080BEI039)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Dr. Binod Sapkota

August, 2026

DECLARATION

We hereby declare that the report of the project entitled **"A Privacy-Preserving Edge TinyML System for Acoustic Comfort Estimation Using Hybrid DSP-CNN Fusion and Visual Feedback"**, which is being submitted to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in **Electronics, Communication and Information Engineering**, is a bonafide report of the work carried out by us.

The materials contained in this report have not been submitted to any University or Institution for the award of any degree, and we are the only authors of this complete work. No sources other than those listed here have been used in this work.

Plan Ghimire (080BEI028) _____

Pranjal Shrestha (080BEI033) _____

Sagun Acharya (080BEI037) _____

Samman Ghale (080BEI039) _____

Date: August, 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a minor project work entitled **"A Privacy-Preserving Edge TinyML System for Acoustic Comfort Estimation Using Hybrid DSP-CNN Fusion and Visual Feedback"** submitted by **Plan Ghimire, Pranjal Shrestha, Sagun Acharya, Samman Ghale** in partial fulfillment for the award of Bachelor's Degree in Electronics, Communication and Information Engineering.

The project was carried out under special supervision and within the time frame prescribed by the syllabus. We found the students to be hardworking, skilled, and ready to undertake related work in their field of study and hence recommend the award of partial fulfillment of the Bachelor's Degree in Electronics, Communication and Information Engineering.

Dr. Binod Sapkota
Project Supervisor
Department of Electronics and Computer Engineering, Thapathali Campus

---

External Examiner

Er. Praches Acharya
Project Co-ordinator
Department of Electronics and Computer Engineering, Thapathali Campus

---

Er. Umesh Kanta Ghimire
Head of the Department
Department of Electronics and Computer Engineering, Thapathali Campus

Date: August, 2026

COPYRIGHT

The author has agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the author has agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited. Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Thapathali Campus.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Thapathali Campus, for providing the academic environment, resources and guidance required to prepare this project progress report.

We are deeply thankful to our supervisor, Dr. Binod Sapkota, for his continuous guidance, valuable suggestions, and encouragement throughout the development of this project and the preparation of this report. His insights have been instrumental in helping us navigate the technical challenges of our implementation.

We also extend our appreciation to our teachers, internal evaluators, and colleagues for their constructive feedback and recommendations during our progress reviews, system refinement, and defence preparation.

This project addresses the practical problem of acoustic disturbance in shared spaces. By implementing embedded technologies such as the ESP32-S3, INMP441 I2S microphone, digital signal processing, embedded deep learning and IoT dashboards, we aim to build a feasible, privacy-preserving and product-oriented system. The feedback received during our progress discussions has been vital in refining our methodology and guiding our next steps toward final implementation.

Project Team

ABSTRACT

This project is a privacy-preserving, edge-based smart acoustic comfort indicator that classifies acoustic behavior in classrooms and shared workspaces into intuitive visual states such as Smooth, Pre-Warning, Warning, Loud, Recovery and Spike. It aims to mitigate auditory distractions in working and learning environments by providing non-judgmental visual feedback on the room's acoustic state. The planned hardware platform consists of an ESP32-S3 microcontroller, an INMP441 I2S MEMS microphone, dual status displays, custom visual indicators, a battery charging circuit, speaker support and Wi-Fi connectivity. The submitted firmware calculates A-weighted loudness, FFT-based DSP features, speech-band energy, peak events and a 49×64 Log-Mel feature tensor locally. A compact, fully quantized INT8 DS-CNN and digital signal processing (DSP) are then used to classify the acoustic state on-device. Only anonymous scalar metrics are exposed through the local dashboard, ensuring user privacy, while MQTT-based remote telemetry remains a planned extension. The expected outcome is a functional, product-level prototype featuring real-time audio analysis, privacy-preserving data processing, remote dashboard visualization and a comprehensive validation report.

Keywords: *A-weighting, Acoustic Comfort, CNN, Embedded Deep Learning, ESP32-S3, FFT, INMP441, Log-Mel, MQTT, Privacy by design, TensorFlow Lite Micro*

TABLE OF CONTENTS

DECLARATION . . . . . **i**

CERTIFICATE OF APPROVAL . . . . . **ii**

COPYRIGHT . . . . . **iii**

ACKNOWLEDGEMENT . . . . . **iv**

ABSTRACT . . . . . **v**

LIST OF FIGURES . . . . . **ix**

LIST OF TABLES . . . . . **x**

LIST OF ABBREVIATIONS . . . . . **x**

1. INTRODUCTION **1**

1.1. Background 1

1.2. Motivation 1

1.3. Problem Definition 2

1.4. Objectives 3

1.5. Scope and Applications 3

1.6. Report Organization 3

2. LITERATURE REVIEW **5**

2.1. Irrelevant Speech, Lombard Effect and Acoustic Comfort 5

2.2. Acoustic Standards, Speech Levels and Threshold Selection 5

2.3. A-Weighting and Digital Signal Processing 5

2.4. Voice Activity Detection and Embedded Audio Features 6

2.5. Transfer Learning and Knowledge Distillation Theory 7

2.6. Embedded Deep Learning and TinyML 8

2.7. IoT-Based Noise Monitoring Systems 11

2.8. IoT Communication and Privacy-Preserving Edge Processing 11

2.9. Comparative Gap Analysis 12

3. REQUIREMENT ANALYSIS **14**

3.1. Introduction 14

3.2. Functional Requirements 14

3.3. Non-Functional Requirements 16

3.4. Hardware Requirements 17

3.5. Software Requirements . . . 18
3.6. Constraints and Assumptions . . . 18

4. SYSTEM ARCHITECTURE AND METHODOLOGY . . . 19
4.1. Overall System Architecture . . . 19
4.2. DSP Signal Processing Pipeline . . . 20
4.3. CNN Model Architecture and Inference . . . 27
4.4. State Transition Architecture . . . 32
4.5. Privacy Boundary Architecture . . . 38
4.6. Dashboard Instrumentation and Diagnostic Telemetry . . . 39

5. IMPLEMENTATION DETAILS . . . 43
5.1. Hardware Implementation . . . 43
5.2. Software Implementation . . . 47
5.3. Calibration Process . . . 53
5.4. Dataset Creation . . . 54
5.5. Integration and Testing . . . 56

6. RESULTS AND VALIDATION . . . 57
6.1. Machine Learning Results . . . 57
6.2. System Performance Results . . . 58
6.3. Acoustic Measurement and Calibration . . . 58
6.4. Physical Prototype Result . . . 59
6.5. Web Dashboard Result . . . 59
6.6. Scenario-Based Validation . . . 60
6.7. Overall Results and Discussion . . . 60

7. DISCUSSION, FEASIBILITY AND FUTURE ENHANCEMENTS . . . 61
7.1. Discussion of Results . . . 61
7.2. Feasibility Analysis . . . 61
7.3. Limitations . . . 62
7.4. Future Enhancements . . . 62

8. CONCLUSION . . . 63

A. Project Budget . . . 64
A.1. Detailed Cost Breakdown . . . 64
A.2. Development and Testing Cost . . . 64

B. Project Timeline . . . 65
B.1. Project Gantt Chart . . . 65

B.2. Major Project Activities 65

C. Final Circuit and Prototype Assembly 67
C.1. Overall Circuit Diagram 67
C.2. Final 3D-Enclosed Prototype 67
C.3. Prototype Assembly 67

D. Hardware Module Specifications 69
D.1. ESP32-S3 N16R8 Development Board 69
D.2. INMP441 MEMS Microphone 69
D.3. TFT SPI Display 69
D.4. WS2812B RGB LED 69
D.5. Prototype Enclosure 70

E. Dataset and Model Development Details 71
E.1. Dataset Classes 71
E.2. Audio Characteristics 71
E.3. Dataset Preparation 71
E.4. Dataset Distribution 71
E.5. Feature Extraction Configuration 72
E.6. Model Development Summary 72
E.7. Dataset Split Summary 72

Bibliography 73

LIST OF FIGURES

Figure 4-1. Overall System Block Diagram 19
Figure 4-2. DSP Signal Processing Pipeline of the Project 21
Figure 4-3. Architecture of the Compact DS-CNN Student Model 29
Figure 4-4. State Transition Diagram 32
Figure 4-5. Standard Score Escalation and Stepped Decay 37
Figure 4-6. Loud State Timing Diagram 37
Figure 4-7. Hardware Fault Lockout Timing Diagram 38
Figure 4-8. Privacy Boundary Architecture 38
Figure 5-1. Enclosure Design- 3D Model 47
Figure 6-1. Final integrated prototype. 59
Figure 6-2. AuraSync local web dashboard showing real-time system information. 60
Figure B-1. Gantt chart of the project 65
Figure C-1. Main circuit diagram of the prototype 67

LIST OF TABLES

Table 2-1. Comparative Gap Analysis of Prior Works and Proposed System . . 13
Table 4-1. Compact DS-CNN Student Architecture 29
Table 4-2. Fused Disturbance Index Components 33
Table 4-3. FSM Waterfall Priority Transition Rules 36
Table 4-4. Privacy Boundary: What Is and Is Not Exposed 39
Table 4-5. Diagnostic Metrics Exposed Through Dashboard 40
Table 5-1. ESP32-S3 Hardware Pin Mapping and Interface Allocation 43
Table 5-2. I2S Initialization Configuration 44
Table 5-3. WS2812B Timing Parameters 46
Table 5-4. Firmware Task Allocation and Core Partitioning 48
Table 5-5. Real-Time Audio Processing Pipeline 49
Table 5-6. Acoustic State Transition Conditions 49
Table 5-7. Counter Management with Hysteresis 50
Table 5-8. CNN Inference Processing Pipeline 51
Table 5-9. Visual Feedback Logic 52
Table 5-10. Dashboard Status Parameters 52
Table 5-11. Calibration Results 53
Table 5-12. Baseline Adaptation Parameters 54
Table 5-13. Dataset Distribution 55
Table 5-14. Data Augmentation Techniques 55
Table 5-15. Feature Extraction Parameters 56
Table 6-1. TinyML Model Performance 57
Table 6-2. Confusion Matrix of the Full INT8 Quantized Model on the Test Split 57
Table 6-3. Class-wise Performance of the INT8 Model 58
Table 6-4. Memory Usage 58
Table 6-5. Calibration Verification Results 58
Table 6-6. Scenario-Based Validation Results 60
Table A-1. Total Hardware and Prototype Cost 64
Table D-1. Essential ESP32-S3 N16R8 specifications 69
Table D-2. Essential INMP441 specifications 69
Table D-3. Essential TFT display specifications 69
Table D-4. Essential WS2812B specifications 69
Table E-2. Dataset audio characteristics 71
Table E-4. Dataset distribution 72
Table E-7. Summary of the dataset and model-development pipeline 72

LIST OF ABBREVIATIONS

ADC	Analog to Digital Converter
CNN	Convolutional Neural Network
dBA	A-weighted decibel
DMA	Direct Memory Access
DSP	Digital Signal Processing
FFT	Fast Fourier Transform
GPIO	General Purpose Input Output
I2C	Inter-Integrated Circuit
I2S	Inter-IC Sound
IoT	Internet of Things
LAeq	Equivalent Continuous A-weighted Sound Level
LCD	Liquid Crystal Display
LED	Light Emitting Diode
MFCC	Mel-Frequency Cepstral Coefficients
MQTT	Message Queuing Telemetry Transport
OLED	Organic Light Emitting Diode
PCM	Pulse-Code Modulation
RAM	Random Access Memory
RMS	Root Mean Square
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
TinyML	Tiny Machine Learning
TFLM	TensorFlow Lite for Microcontrollers
UI	User Interface
Wi-Fi	Wireless Fidelity

1. INTRODUCTION

1.1. Background

Offices and classrooms are environments where there are many people huddled together in a single room or a close environment. While it is necessary to maintain a quiet environment so everyone else in the room can focus without any distractions, this may not be fully possible due to it being hard to gauge how loud one is being, especially in already noisy environments where background noise covers up speech.

Human interaction is also usually necessary in these environments in order to share ideas and amplify the understanding of whatever is being discussed. This introduces a practical problem: how to recognize how loud one is being and how to mitigate the noise without creating more noise in turn or passing on an authoritative tone that may create a stressful environment unnecessarily.

The performance of people working in any shared environment with task-irrelevant speech, that is, speech that may not be relevant but is nonetheless present intelligibly in the background, decreases significantly even more than continuous noise at the same sound level [1]. The Speech Transmission Index measures speech intelligibility, and an index above 0.6 indicates that the semantics are largely understandable. Haka et al. observed that long-term memory recall decreased in conditions with higher speech intelligibility [2]. Irrelevant speech was also observed to increase subjective workload as well as overall stress of the people subjected to it [3]. The cognitive load generated by this processing is highly dependent on the linguistic structure of the noise, a phenomenon known in cognitive psychology as the Irrelevant Speech Effect.

In shared spaces, there can be presence of numerous feedback loops relating to human speech. The Lombard effect, first documented by Étienne Lombard in 1911, describes the involuntary and subconscious tendency of people to increase the level of their speech in a noisy environment in order to keep speech intelligible [4]. It has been observed that the Lombard effect can be triggered when environmental noise increases, leading speakers to increase their speech level by several decibels for every 10 dB increase in background noise [5, 6].

Especially in shared spaces, a feedback loop is observed between multiple speakers who need to speak simultaneously to maintain intelligibility for their respective conversations.

1.2. Motivation

Current monitoring methods for noise levels include active human policing, commercial numerical decibel meters or loud auditory alarms. These methods introduce various

problems such as authoritative and social friction, demanding focal attention, or creating more stress and noise.

Traditional sound meters provide a numerical value for the noise being created in an environment. These devices are mainly created as engineering tools and feature rapidly changing abstract values. A person in a working environment cannot cognitively direct their focus onto the numerical value without breaking complete focus on the current task. As a result, these devices can become another source of distraction rather than an effective measure to reduce noise.

Many commercial noise warning systems try to solve constant monitoring through addition of a buzzer component that warns users when the noise level becomes too high. However, from a psychoacoustic standpoint, this approach is flawed and counterproductive because it adds sound to an already loud room. A buzzer may startle occupants, increase stress and break concentration, which is the opposite of the project objective.

To mitigate these issues and make sure the outcome is productive, this project aims to utilize slow, adaptive and color-shifting LED animations. This is done to leverage the human peripheral visual system, which is optimized to detect changes in color, motion and light intensity over a wide field of view. Such visual feedback requires much less cognitive concern than constantly interpreting changing numerical values and is less intrusive than a buzzer.

1.3. Problem Definition

The main problem this project tries to solve is the noise problem in classrooms as well as shared workspaces. The Lombard effect states that noise created in order to combat already existing noise levels is usually involuntary and subconscious. This means that participants may not be aware of the noise they are creating themselves.

The project also aims to reduce the paradoxical cascading noise sources that are created while trying to reduce previous noise. Usually, supervisors or peers in the area need to speak up and increase the noise to make sure the noise level is decreased. In addition, a person commanding others to reduce noise may put forward an authoritative tone and may induce or increase stress on the people who were unaware of the noise being made.

Currently, there is a lack of an objective, real-time mechanism to break the subconscious feedback loop. While human supervisors are able interfere, their authority can be a stress inducer and may also disrupt the flow of the shared space. The fundamental problem in these scenarios is the absence of an objective, non-judgmental and non-human arbitrator in shared spaces.

By depersonalizing the feedback loop, an automated visual indicator serves as an impartial, non-human mediator. Because the system is objective and treats all occupants equally, it removes the personal judgment and defensive triggers associated with human-to-human policing, allowing occupants to self-regulate their volume naturally and maintain a collaborative, low-stress environment.

1.4. Objectives

To ensure that the problems are addressed correctly, the following objectives need to be fulfilled:

- To design, develop and validate a privacy-preserving edge-based smart acoustic comfort indicator based on a hybrid processing pipeline consisting of digital signal processing and custom embedded deep learning.
- To provide real-time, non-invasive visual feedback as well as anonymous cloud analytics for acoustic comfort in classrooms and workplaces.

1.5. Scope and Applications

The project aims to be an integration of hardware, digital signal processing, embedded deep learning and dashboard visualization. The audio is captured using an INMP441 I2S MEMS microphone and it the audio is processed by two different halves, a deterministic DSP process and a probabilistic CNN process. Both the results from the DSP as well as the CNN halves are combined into a single metric which determines the state of the surrounding noise environment. The visual feedback will be delivered through a TFT display as well as RGB LED strips which is placed in a suitable enclosure.

The application of this project is ranges from libraries, classrooms, and quiet study spaces to corporate offices and collaborative zones.

Any environment where concentration and comfort unhindered from noise is where this device will be applicable. This device is simply a non-intrusive indicator of the audio levels and not a legally standard sound level meter. It processes the audio on the embedded device itself without any external storage of the recorded audio. A local web dashboard is maintained, designed to operate in classrooms and offices, it requires a Wi-Fi capable client.

1.6. Report Organization

This report is organized in terms of following chapters mentioned below:

- **Chapter 1: Introduction** – It explain about the background, motivation, problem definition, objectives and scope of the project.
- **Chapter 2: Literature Review** – It reviews existing research on acoustic comfort, DSP, embedded deep learning, IoT noise monitoring and privacy-preserving systems.
- **Chapter 3: Requirement Analysis** – It describes about the hardware, software requirements and component's selection for the project.
- **Chapter 4: System Architecture and Methodology** – It explains the overall system architecture, block diagram, mathematics, algorithms and methods used in the project.
- **Chapter 5: Implementation Details** – It provides detailed implementation of hardware interfacing, calibration, dataset creation, and firmware development.
- **Chapter 6: Results and Analysis** – It presents the outputs, validation results, error analysis and comparisons with expected/reference values.
- **Chapter 7: Future Enhancements** – It discusses about the potential technical improvements.
- **Chapter 8: Conclusion** – It summarizes the project achievements and findings.
- **Appendices** – It includes project budget, timeline, circuit diagrams, PCB designs, module specifications, and dataset details.
- **References** – It lists all references used in project in IEEE format.

2. LITERATURE REVIEW

2.1. Irrelevant Speech, Lombard Effect and Acoustic Comfort

Acoustic comfort does not equate to silence. A productive acoustic environment in an office or classroom is a space where the sound level of speech and activities matches the function of the room. While complete silence may not be applicable to a shared space, intelligible background speech can be disruptive during focused work. Research on the irrelevant speech effect has found that intelligible background speech distracts from memory processes and negatively affects performance even when the overall sound level is relatively low [1–3].

In this case, the Lombard effect serves as a physiological explanation of how a room can gradually get louder. With the increase in the amount of background sound, speakers begin to automatically speak louder to keep the intelligibility level [4–6]. In multi-speaker environments, this becomes a feedback loop: one loud group causes others to speak louder, which increases the background level and encourages further vocal effort. This loop is a key reason why this project focuses on gentle feedback rather than direct punishment or alarms.

2.2. Acoustic Standards, Speech Levels and Threshold Selection

As for the acoustic threshold implemented in this project, it is justified by the existing guidelines on acoustic comfort in classrooms and office settings. ISO 22955:2021 defines the quality of sound in open offices and highlights speech disturbance, discretion, intelligibility, and the Lombard effect in the context of shared workplaces [7]. Similarly, the strict guidelines in ANSI/ASA S12.60 are aimed at creating a comfortable acoustic environment for education [8].

This project does not seek to replace the use of a Class 1 or Class 2 sound level meter. Hence, the threshold values are applied as operating limits for a behavioral marker. Human conversations at approximately one meter generally measure around 55 to 65 dBA, while raised voices can reach 70 dBA [9]. As such, 55–70 dBA can form the basis for the conversational warning zone, and greater than 70 dBA can be applied for louder speech state limits. Additionally, this is done with CNN confidence to prevent false alarms on other sources generating the same level of sound.

2.3. A-Weighting and Digital Signal Processing

For testing environmental noise, measurement should reflect human hearing perception. Raw microphone signals may overrepresent low-frequency sources such as air-conditioning systems, ventilation units and traffic rumble. Rimell et al. developed

implementation approaches for A-weighting and C-weighting filters using international acoustic standards [10]. A-weighting reduces very low and very high frequencies while emphasizing the range where the human ear is more sensitive, making it a common basis for environmental and occupational noise measurement.

However, this process is simply a method of applying a frequency response. It cannot differentiate between speech and non-speech sounds. Two sounds can be of equal intensity but come from different sources, such as conversations, typing, hand claps and machine operation. Consequently, this project implements A-weighting alongside FFT analysis, speech-band analysis, Log-Mel features and TinyML classification.

The Fast Fourier Transform is an efficient algorithm that converts time-domain signals into frequency-domain signals. The algorithm proposed by Cooley and Tukey minimizes the computational complexity of directly computing the Fourier transform [11]. Most workplace speech occupies the bandwidth from 300 Hz to 3400 Hz, while many steady noise sources occupy lower frequencies. This project uses FFT-derived speech-band metrics and a Log-Mel feature set to help distinguish conversations from low-frequency background noise.

2.4. Voice Activity Detection and Embedded Audio Features

2.4.1. Log-Mel Spectrogram vs. MFCC Feature Selection

In order to compress the audio features in any dataset, usual speech processing methods apply Mel-Frequency Cepstral Coefficents (MFCCs). Generating these MFCCs require using and applying a discrete cosine transform to a different type of spectrogram called Log-Mel spectrogram. The purpose of this transformation is to remove the correlation in between the different frequency bands. But while decorrelation is helpful for usage in simple linear classifiers, it does more harm than good for Convolutional Neural Networks (CNNs) [12]. Using a DCT can remove the spatial structure inherent in the dataset which is a requirement for CNNs as they work by looking through and recognizing these spatial correlations.

2.4.2. Feature Extraction Geometry and Input Dimensionality

The input features map to a fixed 2D grid. The sample size of the signal is 16 kHz mono [9]. A 512-point Fast Fourier Transform (FFT) runs using a Hann window with a 320-sample hop length [11]. The final power spectrum is 64 Mel bins spanning 80 Hz to 7600 Hz. Over a 1-second audio window (16,000 samples) [13], the exact number of time frames is:

$$\text{Time_Frames} = \left(\frac{N_{\text{samples}} - N_{\text{mf}}}{N_{\text{hop}}}\right) + 1 = \left(\frac{16000 - 512}{320}\right) + 1 = 49 \tag{2-1}$$

A 2D input tensor of $[49 \times 64 \times 1]$ is created at the end for every 1-second audio frame.

2.4.3. Per-Clip Mean and Per-Mel Normalization

A scalar mean subtraction which is followed by a standardization for each Mel bin is applied in order to reduce the microphone gain and the physical distance.

$$\mu_{\text{clip}} = \frac{1}{T \cdot F} \sum_{t=1}^{T} \sum_{f=1}^{F} X_{\text{logmel}}(t, f) \tag{2-2}$$

$$X_{\text{norm}}(t, f) = \frac{X_{\text{logmel}}(t, f) - \mu_{\text{clip}} - \mu_f}{\sigma_f} \tag{2-3}$$

Here, μ_{clip} is the scalar mean of the spectrogram for all the 49 frames and 64 Mel bins. μ_f and σ_f are the training set statistics for each Mel bin.

2.5. Transfer Learning and Knowledge Distillation Theory

2.5.1. The Transfer Learning Paradigm in Audio

Training deep learning models from scratch requires massive datasets. The network needs thousands of hours of audio just to learn basic acoustic properties like pitch, timbre, and temporal envelopes. For custom embedded applications, such datasets are not available. Transfer learning solves this by taking a model pre-trained on a massive dataset and adapting its learned representations to the target task [14].

YAMNet is a MobileNetV1 architecture trained on millions of AudioSet clips [12]. The YAMNet model serves as the foundation.For each 1-second clip, it extracts 1024-dimensional embeddings [12]. It calculatins and concatenates the mean and standard deviation of these embeddings along the time axis yields a 2048-dimensional feature vector. A lightweight classification head trained on these vectors achieves 96.06% accuracy on the test split. This bypasses the need for a massive local training dataset.

2.5.2. The Teacher-Student Knowledge Distillation Framework

It is not feasible to run YAMNet dirrectly on an embedded microcontroller. So, Knowledge Distillation transfers the predictive power of this heavy "teacher" model to a compact "student" model [14].

Then the student is trained to match the softened probability outputs of the teacher rather than just hard binary labels. The teacher's final Softmax layer is softened using a temperature scaling parameter $T = 2.0$:

$$P_i = \frac{e^{\frac{z_i}{T}}}{\sum_j e^{\frac{z_j}{T}}} \tag{2-4}$$

where z is the logit for class i. The hidden similarities between classes is revealed by the softening process. This shows that the student that has a specific speech clip also shares acoustic similarity with non-speech noise. Providing this soft target distribution prevents the student from overfitting and allows a highly compressed model to generalize well on inexpensive hardware [15].

2.6. Embedded Deep Learning and TinyML

Tiny Machine Learning (TinyML) is a field of machine learning used for the deployment of machine learning models in embedded devices such as microcontrollers. Conventional machine learning systems highly depend on cloud servers or high-performance processors, while TinyML enables inference to be performed directly on edge devices with limited memory, processing capability, and power availability [14]. This type of approach is particularly suitable for continuous sensing applications. The data can be processed locally without requiring constant network communication.

Convolutional Neural Networks (CNNs) can learn spatial and temporal patterns from two -dimensional audio representations such as spectrograms and Log-Mel spectrograms. So, they are widely used for acoustic applications. Conventional CNN architectures can require significant memory and computation. That makes direct deployment on microcontrollers difficult. Therefore, for embedded audio classification, lightweight neural network architectures are required.

2.6.1. Depthwise Separable Convolution for Embedded Audio Classification

The best alternative to conventional convolutional networks is Depthwise Separable Convolution (DS-CNN). It does not perform spatial filtering and channel combination simultaneously; instead, it separates the operation into two stages. A 1×1 pointwise convolution then combines information across the channels [15].

This separation, done by the DS-CNN architecture, helps in reducing the number of model parameters and multiplication operations in comparison to the standard convolution, while the ability of the network to learn acoustic features remains intact.

As a result, DS-CNN architectures are suitable for embedded audio recognition systems where processor speed, RAM, flash memory, and power consumption are limited [15].

Speech and environmental acoustic classification are especially relevant to lightweight CNN architectures. Since acoustic information can be represented as a two-dimensional time-frequency feature map, convolutional filters can therefore identify local patterns corresponding to spectral energy distributions, temporal changes, and other acoustic characteristics. The use of such architectures allows these features to be classified locally instead of transmitting raw audio to external computing systems.

2.6.2. Model Quantization for TinyML

Lightweight architectures reduce computational requirements. However, trained neural networks commonly use 32-bit floating-point parameters. When the floating-point model is deployed on a microcontroller, it can still consume a considerable amount of memory and computational resources. Therefore quantization is used to reduce the numerical precision of neural network weights and activations [14].

Post-Training Quantization (PTQ) converts a previously trained floating-point model into a lower-precision representation without requiring the model to be retrained. The network weights and activations can be represented using signed 8-bit integer values in full integer quantization. A real-valued quantity r can be mapped to its corresponding quantized value q using a scale factor S and zero-point Z:

$$q = \text{round}\left(\frac{r}{S}\right) + Z \tag{2-5}$$

The scale controls the relationship between integer ranges and floating-point. The zero-point determines the integer value corresponding to real-valued zero. The representative input samples are generally used to estimate essential activation ranges for the network in the quantization process [14].

INT8 quantization can significantly reduce the model storage requirements compared to that of 32-bit floating-point representations. However, quantization may introduce a reduction in numerical precision. Therefore, the performance of the quantized model should be evaluated against the original floating-point model to determine whether the reduction in model size and computational cost introduces an acceptable change in classification performance.

2.6.3. TensorFlow Lite Micro for Edge Inference

TensorFlow Lite Micro (TFLM) is an execution engine designed specifically to execute machine learning inference on microcontrollers and other devices with restricted computational resources [14]. It is built for low-resource environments. It does not require an OS or dynamic memory allocation. Unlike conventional TensorFlow environments, embedded inference frameworks operate without requiring a full operating system and are designed to work within predetermined memory constraints.

TinyML-based acoustic device transforms the acquired audio signal into suitable features on the embedded device before being passed to a compact neural network. The network produces a classification result locally. This enables the system to react to acoustic conditions without transmitting raw audio to a server.

This feature is important from a privacy perspective. Shared spaces and enviroments may contain some sensitive or private information. Local inference reduces the need to transmit or permanently store raw audio. The required information, such as acoustic state or numerical sound level indicators, is exposed to the monitoring systems.

2.6.4. Relevance of TinyML to Acoustic Monitoring

Previous research in TinyML shows that it is possible for compact neural networks to perform acoustic event detection and classification on resource-constrained hardware like microcontrollers [15, 16]. So TinyML is very suitable for systems with low latency and limited computing resources.

Sound level alone is not sufficient to determine the context or source of an acoustic event. Embedded machine learning is perfect for this purpose. In environments, various sounds with similar measured intensity may represent different conditions. Mechanical noise, human conversation, and many other environmental events justify this situation. Such acoustic events need to be classified properly. Therefore, lightweight embedded classification with conventional acoustic measurements provides a sufficient basis for more context-aware noise monitoring.

The prior work on TinyML validates the use of lightweight convolutional networks, quantization, and embedded inference for acoustic classification. The aforementioned topics, such as neural network architecture, training procedure, hardware deployment, and quantization configuration in this study, are described separately in the Research Methodology chapter.

2.7. IoT-Based Noise Monitoring Systems

Marques and Pitarma proposed an economical IoT noise logger solution for real-time monitoring of acoustic comfort. It is based on sound level sensing and Wi-Fi dashboards, yet without applying machine learning and speech-noise separation [17]. It illustrates the application of IoT dashboards for acoustic comfort, and also highlights the problem of dBA-only monitoring.

Hammad et al. showed the applicability of TinyML and anomaly detection algorithms to detect acoustic events in near-zero latency and low power consumption on microcontrollers [16]. Research of this kind gives ground for applying learning on devices in order to process data privately. But, the problem of anomaly detection does not tell us what kind of sound leads to that anomaly. This project also classifies the acoustic states into several classes and employs a visual feedback mechanism.

2.8. IoT Communication and Privacy-Preserving Edge Processing

Internet of Things (IOT) systems can also work for lightweight communication mechanisms. This helps transfer sensor measurement data from embedded devices to other devices such as dashboards and many other devices. Message Queuing Telemetry Transport (MQTT) communication is widely used for such applications. In the publish-subscribe communication model, devices publish data to specific topics through a broker. Interested clients subscribe to those topics to receive the transmitted information. MQTT follows the publish-subscribe communication model [18, 19].

The publish-subscribe architecture helps in reducing the direct dependency between data producing devies and receiving applications. MQTT uses lightweight message headers and persistent communication sessions. It is also useful for distributed IoT systems where multiple sensors or monitoring interfaces may need to exchange information efficiently [18].

MQTT is appropriate for resource-constrained embedded systems since it is a lightweight communication protocol. Instead of repeatedly establishing independent HTTP request-response transactions, MQTT can provide efficient communication between edge devices, brokers, and remote monitoring applications.

2.8.1. Privacy Considerations in Acoustic Monitoring

Microphones may capture sensitive information. To prevent this from happening, acoustic monitoring systems introduce important ethical and privacy considerations. Transmitting or permanently storing raw audio may expose sensitive information.

Therefore, privacy should be considered important in embedded acoustic monitoring systems.

In a privacy-preserving edge computing approach, signal processing and machine learning inference are performed locally on the embedded device rather than transmitting audio to a remote server. [20]. Privacy-preserving edge computing is best alternative to cloud-based audio processing. The required information such as classification results, numerical acoustic indicators or event statistics, need to be communicated to external systems.

This approach of processing audio locally on edge devices reduces both network requirements and the amount of sensitive information. Temporary audio buffers may be maintained only for the duration required for feature extraction and inference, after which the raw samples can be discarded. This approach follows the principle of data minimization by limiting communication and storage to information required by the application.

2.8.2. Edge Intelligence and Privacy-by-Design

Functions such as local signal processing, restricting remote access, not storing permanent raw audio data, and transmitting derived numerical information are essential parts of an intelligent acoustic system. Privacy-by-design approaches integrate privacy and ethical considerations into the architecture of edge systems rather than applying them only after deployment. [20].

Combining TinyML with privacy-preserving edge processing is appropriate for shared workspaces where there might be a flow of sensitive environmental information. A lightweight neural network performs local inference. MQTT transmits the resulting non-audio information required for monitoring and visualization.

The literature therefore supports an architecture in which acoustic analysis is performed locally, and lightweight IoT communication is used for transmitting derived monitoring information. The specific communication protocol, transmitted variables, and privacy controls mentioned in the proposed system are thoroughly described in the Research Methodology and System Design chapters.

2.9. Comparative Gap Analysis

Apart from previous work limitations, this project solves the limitation of prior research through a practical hybrid solution approach. Simple dBA logger devices record sound levels but are unable to detect whether human voice is mixed with fans, AC, door closure, or other noises in the workspace. Also, binary detectors can identify anomalies but lack

state separation functionality that would be required by the indicator. Speaker recognizer devices utilize ESP32-S3 and INMP441 audio sensor capabilities; however, their purpose is speaker identification rather than environmental sound state feedback. This project combines DSP, TinyML, state-transition hysteresis, privacy preserving dashboard and visual feedback merged into a low-cost prototype.

Table 2-1. Comparative Gap Analysis of Prior Works and Proposed System

System / Prior Work	**Reference(s)**	**Processing Approach**	**Main Strength**	**Limitation / Research Gap**
Traditional decibel meter / sound-level measurement	[9, 10]	Measures RMS or A-weighted sound level and displays a dB/dBA value.	Provides simple and objective sound-level measurement and calibration.	Cannot identify sound type and does not provide speech/noise separation or adaptive feedback.
IoT-based noise monitoring system	[17–20]	Measures sound level and sends processed information to a dashboard or remote platform.	Supports real-time monitoring, logging, and visualization.	Mostly dBA-based, with limited local intelligence; privacy depends on transmitted data.
TinyML acoustic anomaly / audio event detector	[14–16, 21]	Uses lightweight embedded ML models for acoustic event or anomaly detection.	Supports low-power and low-latency inference directly on embedded devices.	Usually focuses on event detection without stable acoustic comfort states or behavioral feedback.
This project's proposed system	This work; supported by [14, 20]	Combines DSP and INT8 CNN/TinyML processing on ESP32-S3 with a local HTTP dashboard.	Integrates acoustic metrics, sound classification, visual feedback, and privacy-preserving local processing.	Requires calibration, dataset refinement, and further real-world validation.

3. REQUIREMENT ANALYSIS

3.1. Introduction

The main objective of requirements analysis is to identify the expected operations of the system and the conditions required to complete it perfectly. Both hardware and software requirements must be considered due to acoustic sensing, local monitoring, embedded machine learning, digital signal processing, and visual feedback.

This chapter presents the functional requirements, non-functional requirements, hardware requirements, software requirements, and major constraints considered during the development of the proposed system.

3.2. Functional Requirements

Functional requirements describe the major operations that the system should perform during normal use.

3.2.1. Audio Acquisition

The system should continuously hear and capture the environmental or background sound with the help of a microphone. The microphone should be able to provide sufficient information for acoustic sound classification and feature extraction. The sampling process should preserve important frequency components.

3.2.2. Sound Level Estimation

The acoustic level of audio captured from the microphone should be estimated. The sound level estimation should not be solely done on the basis of raw digital amplitude; rather, it should be done through Mel-scale conversions. The calibration should be done properly so that the estimated sound level can be easily adjusted and compared using a reference measurement device.

3.2.3. Acoustic Feature Extraction

In the environment, there are many sounds that have identical intensity but may differ in acoustic characteristics. So the use of multiple acoustic features is important. Necessary and relevant features should be extracted from the incoming audio signal to identify the acoustic condition of surrounding.

3.2.4. Sound Classification

The system should be able to identify and classify general acoustic conditions using a lightweight machine-learning model. The main operation of the system should not

depend on continuous communication with an external power or cloud server. So, the classification process should operate directly on the embedded device.

3.2.5. Hybrid Decision Processing

A sound-level measurement alone may not provide sufficient information about the source of a sound. Similarly, a machine learning result alone may not respond quickly enough to abrupt changes in acoustic level. So, the combination of both sound-level measurement and machine learning results is considered. Therefore, the final decision should consider both approaches before determining the current acoustic condition.

3.2.6. Acoustic State Determination

The processed information should be converted into understandable acoustic states. The system should be able to differentiate between normal conditions, increasing disturbances, recovery conditions, sustained high sound levels, and sudden short-duration sound events. There might be small fluctuations in sound. But that should not cause unnecessary changes between states.

3.2.7. Visual Feedback

Visual indications are necessary to know the acoustic state of the surrounding environment. The continuous observation of numerical sound values is hectic and boring. The system should display the visual feedback according to the acoustic state of the surrounding environment. It should remain simple and non-intrusive.

3.2.8. Local Monitoring

A local monitoring interface should display useful acoustic information such as acoustic level, detected state, and other processed measurements. It should remain functional even when the internet connection is gone.

3.2.9. Calibration

The system should provide a method for comparing its estimated acoustic level with a reference sound-level measurement. A correction value obtained during calibration should be retained so that the system can continue using the calibrated value after restarting.

3.2.10. Privacy-Preserving Processing

Audio processing should be performed locally whenever possible. Raw audio should not be stored locally or transmitted to external sources because the acoustic information may

contain private speech or sensitive information. Only the relevant information should be made available outside the audio-processing stage.

3.2.11. Error Handling

The system should detect important operating problems such as invalid microphone readings, sensor initialization failure, or unavailable audio input. Suitable status information should be provided so that such problems can be identified during operation, testing, and maintenance.

3.3. Non-Functional Requirements

Non-functional requirements define the expected quality, performance, reliability, and operational behavior of the system.

3.3.1. Performance

The system should quickly respond to the changes happening in the acoustic environment and provide useful real feedback. All the signal processing measurements should be updated frequently with the changing acoustic environment. The machine learning inference operates using a larger audio window. But neither of the processes should interrupt continuous audio acquisition. This is important for the acoustic system to show results efficiently.

3.3.2. Reliability

The system should remain stable during audio acquisition and processing during normal operation. The operation, such as display updates, network activity, and machine learning inference, should not interfere with continuous microphone sampling.

3.3.3. Privacy

Privacy should be considered throughout the operation of the system because microphones may capture human speech and other sensitive environmental information. Raw audio data should remain within temporary processing memory whenever possible and should not normally be stored as audio recordings.

3.3.4. Usability

The system should present information in such a manner that it can be understood without requiring specialist knowledge. The information should be simple and in a readable manner. Visual feedback should clearly indicate changes in the acoustic condition. The monitoring interface should display important features in a simple and readable manner.

3.3.5. Maintainability

The system should be organized in such a manner that there should be no difficulty in future modification and testing. Therefore, the functions such as audio acquisition, signal processing, machine learning interface, visual output configuration, and communication functions should be kept reasonably modular.

3.3.6. Resource Efficiency

The system operates on edge devices with very low memory and limited processing capability. The machine learning model and many other functions and signal processing algorithms should remain within the constrained memory and computational resources.

3.3.7. Offline Operation

The system should not depend on an internet connection. It should continue to work even when the internet is not available. Functions like audio acquisition, acoustic analysis, acoustic classification, and many other processing functions should continue to operate even without an internet connection. This is the main purpose of the proposed system.

3.4. Hardware Requirements

The selected hardware should provide a suitable platform for functions such as audio acquisition, acoustic analysis, signal processing, embedded machine learning inference, visual output, and local communication.

The main hardware requirements include:

- A microcontroller to run machine learning models and perform embedded signal processing.
- A suitable microphone for input audio acquisition.
- For displaying acoustic conditions and necessary information, a suitable display unit.
- Visual indicators such as RGB LEDs for providing easily recognizable feedback.
- Power supply for operating the system or a rechargeable battery for portability.
- USB or serial connectivity for main operations such as firmware installation, debugging, testing, and calibration.
- Wireless communication capability for accessing the local monitoring interface.

The required pin assignments, electrical connections, and component interfacing are distinctly presented in the sections of system design and implementation.

3.5. Software Requirements

Software tools are required to provide the platform for firmware development, machine learning preparation, model conversion, testing, and system monitoring.

The major software requirements include:

- An appropriate embedded development enviroment for writing, compiling and uploading mivrocontroller firmware.
- Python programming language for dataset preparation, signal analysis, model development and analysis.
- TensorFlow or keras machine learning framework for training and testing the acoustic classification model.
- TensorFlow Lite Micro for executing an optimized neural network on the microcontroller.
- Supporting libraries for audio acquisition, digital signal processing, display control, LED control, and wireless communication.
- A web browser for accessing the local monitoring interface.
- Version-control software for maintaining source code, model files, and related project resources.

All the required model training, feature preparation, quantization, conversion, and embedded deployment procedures are described in the methodology section.

3.6. Constraints and Assumptions

The purpose of this system is not only just to provide certfied sound-level measuremnt, but also provide acoustic awareness and behavorial feedback. There accuracy may be affected due to microphone characteristics, enclose design, enviromental conditions and calibration quality. The model architecture must remain suitable for the real time execution becuase machine learning model is limited by memory and processing capability of microcontroller. The proposed is useful for indoor enviroments rather than the large outdoor environments. Larger oudoor environments, speech recognition and intepretation of spoken content are outside the primary scope of the project.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

This chapter presents the overall system architecture, block diagram, and detailed methodology used in the project. The systems uses the hybrid edge-computing architecture that runs locally in ESP32-S3 microcontroller. The system is divided into two parallel paths: a deterministic Digital Signal Processing (DSP) path and a probabilistic Convolutional Neural Network (CNN) path. This two paths are fused through a hybrid state transition architecture which estimate the acoustic state with stable visual feedback.

4.1. Overall System Architecture

The overall system architecture is shown in Figure 4-1. The system begins from the physical input sound field. It may contain speech, fan/AC hum, claps, keyboard clicks, transient drops, or loud group activity. The INMP441 microphone converts these sound pressure waves into 24-bit digital PCM data. It transmit them through I2S to the ESP32-S3. The DMA controller stores samples from microphone in a volatile ring buffer in SRAM i.e raw audio remains only in temporary memory.

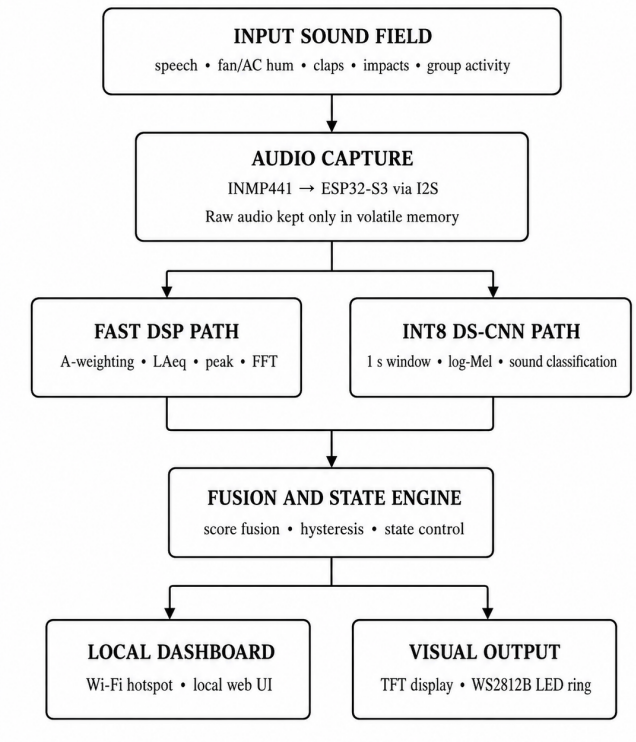


Figure 4-1. Overall System Block Diagram

The system consists of the following main functional blocks:

1. **Acoustic Input:** Physical sound field is captured by the INMP441 microphone. It convertes sound pressure waves to electrical signals.
2. **Signal Acquisition:** I2S audio capture with DMA for efficient data transfer, storing samples in volatile SRAM buffers.
3. **DSP Preprocessing:** Audio preprocessing including DC removal, Hamming window, A-weighting, and FFT analysis.
4. **Feature Extraction:** Generation of acoustic features (loudness, speech-band ratio, crest factor, spectral flux) and Log-Mel spectrogram for CNN input using DSP.
5. **CNN Inference:** Running the quantized INT8 model locally inside microcontroller for sound classification in every 1 second.
6. **Fusion Engine:** Combining DSP metrics and CNN probabilities into a fused disturbance score.
7. **State Management:** Maintaining acoustic states with hysteresis and confirmation timers.
8. **Visual Feedback:** Display output on TFT and LED animations.
9. **Dashboard:** Local web dashboard for status monitoring and diagnostics.

4.1.1. Block Diagram and Interactions

The block diagram in Figure 4-1 shows the interaction between different blocks:

- **Microphone - DMA:** I2S transfers samples directly to memory. It reduces CPU overhead and ensuring continuous audio capture.
- **DMA - DSP Pipeline:** Samples are processed every 100 ms for acoustic feature extraction.
- **DSP Pipeline - Feature Buffer:** Processed features are stored for CNN inference.
- **Feature Buffer - CNN:** For every 1 second, a complete Log-Mel tensor is sent to the CNN for classifying the sound.
- **DSP + CNN - Fusion Engine:** DSP metrics and CNN probabilities are combined.
- **Fusion Engine - State Manager:** State transitions are determined based on fused metrics.
- **State Manager - Visual Output:** Display and LED animations are updated smoothly.
- **State Manager - Dashboard:** Status information is published to the local dashboard.

4.2. DSP Signal Processing Pipeline

The Digital Signal Processing (DSP) pipeline converts the raw digital audio captured from the INMP441 microphone into acoustic and machine learning features. This path is also divided into two paths. The first path is fast DSP path which is used to calculate

acoustics characteristics such as A-weighted sound level, spectrul flux, speed-band energy,etc. The second path is for CNN preprocessing. It converts the 1 second audio into normalized Log-Mel spectrogram for further audio classification.

The overall signal-processing flow is illustrated in Figure 4-2.

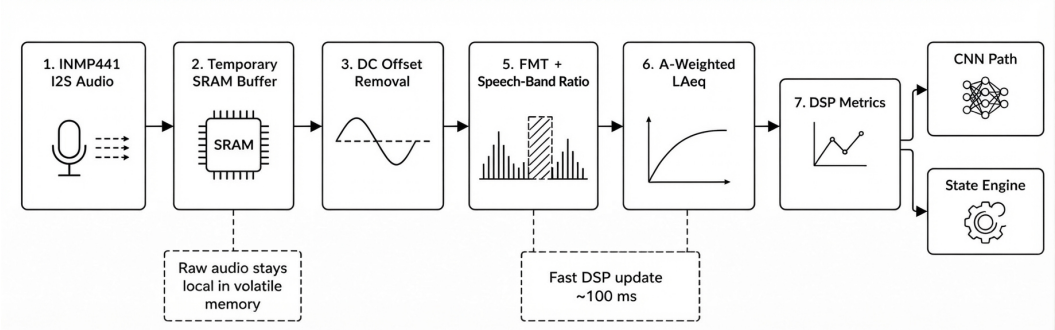


Figure 4-2. DSP Signal Processing Pipeline of the Project

4.2.1. Audio Acquisition and Preprocessing

The system uses an INMP441 digital MEMS microphone which is connected to the ESP32-S3 through the I2S interface. The microphone operates at a sampling frequency of 16 kHz. It provides mono audio samples. The samples are transferred into memory using the I2S and DMA. Hence, no CPU involvement for this process.

The sampling period is given by

$$T_s = \frac{1}{f_s} = \frac{1}{16000} = 62.5\ \mu\text{s}. \tag{4-1}$$

The received audio data are aligned to obtain the valid signed PCM sample. The resulting samples are then passed through the preprocessing stage before being used for the further acoustic calculation

DC Offset and Very-Low-Frequency Suppression

A DC component in the signal can introduce unwanted energy near 0 Hz and affect RMS and FFT calculations. Therefore, a first-order IIR high-pass filter is applied:

$$y[n] = x[n] - x[n-1] + \alpha y[n-1], \qquad \alpha = 0.995, \tag{4-2}$$

where $x[n]$ is the input sample and $y[n]$ is the filtered output.

The approximate cutoff frequency is

$$f_c \approx \frac{1-\alpha}{2\pi T_s}. \tag{4-3}$$

For a sampling frequency of 16 kHz,

$$f_c \approx \frac{0.005}{2\pi(62.5\times10^{-6})} \approx 12.7 \text{ Hz}. \tag{4-4}$$

The low cutoff allows the filter to suppress DC and very-low-frequency components. It retaining the frequency range which is required for speech and environmental sound analysis.

4.2.2. FFT-Based Spectral Analysis

Frequency-domain analysis is used to extract spectral characteristics from the input signal. Before FFT, each analysis frame is multiplied by a Hann window to reduce spectral leakage:

$$w[n] = \frac{1}{2}\left(1-\cos\left(\frac{2\pi n}{N-1}\right)\right), \qquad 0 \le n < N. \tag{4-5}$$

It is transformed into the frequency domain using the discrete Fourier transform:

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1. \tag{4-6}$$

- **Fast DSP path:** A 256-point FFT is used for DSP
- **CNN preprocessing path:** A 512-point FFT is used for Log-Mel spectrogram.

For the 256-point FFT,

$$\Delta f = \frac{f_s}{N} = \frac{16000}{256} = 62.5 \text{ Hz}. \tag{4-7}$$

For the 512-point FFT,

$$\Delta f = \frac{16000}{512} = 31.25 \text{ Hz}. \tag{4-8}$$

This separation allows DSP to run faster and provide higher-resolution for the CNN.

4.2.3. A-Weighted Sound-Level Estimation

A-weighting is applied to approximate the frequency sensitivity of human hearing. The unweighted acoustic measurement gives equal importance to all frequencies which are not in range of human hearing also. So, the A-weighting is used to select only human hearable frequency ranges.

The A-weighting filter is implemented as cascaded second-order sections using a biquad structure. A general biquad section is represented by

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]. \tag{4-9}$$

Here, b_0, b_1, and b_2 are feed-forward coefficients, while a_1 and a_2 are feedback coefficients.

After filtering, the A-weighted RMS value over N samples is calculated as

$$V_{\mathrm{RMS,A}} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} y^2[n]}. \tag{4-10}$$

The RMS value is still a digital-domain quantity.Therefore, it requires calibration before interpreted as a physical sound level.

4.2.4. Calibration and Conversion to L_{Aeq}

The relationship between microphone output and physical sound pressure depends on microphone sensitivity, digital scaling, gain, and the complete signal-processing chain. Hence, the digital RMS value cannot be directly interpreted as dBA without calibration.

The digital level relative to the selected full-scale reference can be expressed as

$$L_{\mathrm{dBFS}} = 20\log_{10}\left(V_{\mathrm{RMS,A}}\right). \tag{4-11}$$

A calibration offset C_{cal} is obtained experimentally by comparing the with a reference sound-level meter under controlled acoustic conditions. The calibrated A-weighted equivalent continuous sound level is then represented as

$$L_{Aeq,T} = 20\log_{10}\left(V_{\mathrm{RMS,A}}\right) + C_{\mathrm{cal}}. \tag{4-12}$$

Here, T represents the measurement interval. The detailed calibration procedure and experimental results are presented in the validation chapter.

4.2.5. Speech-Band Analysis

Only measuring the sound level cannot provide the sufficient information to distinguish the speech activity from other environmental noises. So, we calculate the speech band ratio.

The speech band is defined approximately from 300 Hz to 3400 Hz. For the 256-point FFT at a sampling frequency of 16 kHz,

$$\Delta f = 62.5 \text{ Hz}. \tag{4-13}$$

The lower FFT-bin limit is

$$k_{\text{low}} = \left\lfloor \frac{300}{62.5} \right\rfloor = 4. \tag{4-14}$$

The upper FFT-bin limit is

$$k_{\text{high}} = \left\lceil \frac{3400}{62.5} \right\rceil = 55. \tag{4-15}$$

The speech-band power is calculated as

$$P_{\text{speech}} = \sum_{k=k_{\text{low}}}^{k_{\text{high}}} |X[k]|^2. \tag{4-16}$$

The total positive-frequency power is

$$P_{\text{total}} = \sum_{k=0}^{N/2} |X[k]|^2. \tag{4-17}$$

The speech-band ratio is therefore

$$R_{\text{speech}} = \frac{P_{\text{speech}}}{P_{\text{total}}}. \tag{4-18}$$

A higher speech-band ratio shows that a greater proportion of the measured spectral energy lies within the speech-dominated frequency range. This feature is used together with other acoustic features. It is not treated as a standalone speech classifier.

4.2.6. Spectral Flux and Transient Activity

Spectral flux measures the change in spectral content between two consecutive analysis frames. It is useful for detecting sudden acoustic changes.

Let $X_t[k]$ denote the magnitude spectrum of the current frame and $X_{t-1}[k]$ denote the magnitude spectrum of the previous frame. The spectral flux can be calculated as

$$F_t = \sqrt{\sum_k \left(\hat{X}_t[k] - \hat{X}_{t-1}[k]\right)^2}, \tag{4-19}$$

where $\hat{X}_t[k]$ is the normalized magnitude spectrum of the current frame.

A larger value of F_t indicates a stronger change in spectral content between two consecutive frames.

4.2.7. CNN Feature Extraction

The DSP path is a deterministic path which allows low latency method of obtaining the acoustic information. The CNN half uses a spectral representation to learn more complex patterns in the audio. One second of audio is converted into a 49×64 normalized Log-Mel representation in the CNN part.

Signal Framing

The sampling frequency of the audio signal is around 16 kHz. A 512-sample FFT window and a 320-sample hop length are used.

For a one second input containing 16000 samples, the number of frames is

$$T = \left\lfloor \frac{N_{\text{samples}} - N_{\text{FFT}}}{N_{\text{hop}}} \right\rfloor + 1 = \left\lfloor \frac{16000 - 512}{320} \right\rfloor + 1 = 49. \tag{4-20}$$

For every frame, the power spectrum is calculated as

$$P(t, k) = |X(t, k)|^2. \tag{4-21}$$

Mel-Scale Projection

The 64 mel-frequency bins are spanned throughout 80 to 7600 Hz.

$$\mathrm{Mel}(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right). \tag{4-22}$$

A triangular filterbank is used for Mel projection:

$$M(f,m) = \begin{cases} \dfrac{f - f_{\mathrm{low}}(m)}{f_{\mathrm{center}}(m) - f_{\mathrm{low}}(m)}, & f_{\mathrm{low}}(m) \le f \le f_{\mathrm{center}}(m), \\ \dfrac{f_{\mathrm{high}}(m) - f}{f_{\mathrm{high}}(m) - f_{\mathrm{center}}(m)}, & f_{\mathrm{center}}(m) \le f \le f_{\mathrm{high}}(m), \\ 0, & \text{otherwise.} \end{cases} \tag{4-23}$$

The energy in each Mel band is calculated using

$$\mathrm{Mel}(t,m) = \sum_{k=0}^{N_{\mathrm{FFT}}/2} P(t,k) M(f_k, m). \tag{4-24}$$

Log-Mel Transformation

The logarithmic transformation compresses the dynamic range of the Mel energies:

$$X_{\mathrm{LogMel}}(t,m) = \log\left(\mathrm{Mel}(t,m) + \epsilon\right), \tag{4-25}$$

where $\epsilon = 10^{-9}$ prevents numerical problems caused by a zero input.

The result is 49 time frames and 64 mel bins:

$$\mathbf{X}_{\mathrm{LogMel}} \in \mathbb{R}^{49\times 64}. \tag{4-26}$$

Feature Normalization

Feature normalization done during the training process is similarly applied to the final hardware implementation as well. Firstly, the mean of the log mel is calculated:

$$\mu_{\mathrm{clip}} = \frac{1}{TF} \sum_{t=1}^{T} \sum_{m=1}^{F} X_{\mathrm{LogMel}}(t,m), \tag{4-27}$$

where $T = 49$ is the number of frames and $F = 64$ is the number of Mel bands.

The normalized feature is then calculated using the exported training-set statistics:

$$X_{\text{norm}}(t, m) = \frac{X_{\text{LogMel}}(t, m) - \mu_{\text{clip}} - \mu_m}{\sigma_m}, \tag{4-28}$$

where μ_m and σ_m are the stored mean and standard deviation for the m-th Mel band.

So the final CNN input has the following dimensions:

$$49 \times 64 \times 1. \tag{4-29}$$

4.3. CNN Model Architecture and Inference

The model architecture consists of a teacher and a student model. The student model is designed to be deployable on the ESP32-S3.

4.3.1. Teacher-Student Knowledge Distillation

The teacher model, based on YAMNet, uses a MobileNetV1 based architecture. The model is trained on the AudioSet dataset [**?**]. YAMNet is mainly used in a frozen form, extracting the features during the teacher development step. This model is not used in the ESP32-S3. A second of audio produces a 1024 dimension embedding.

A 2048 dimensioned vector is formed using the embeddings and their mean and standard deviation.

$$\mathbf{f}_{\text{teacher}} = \left[\boldsymbol{\mu}_{\text{YAMNet}} \| \boldsymbol{\sigma}_{\text{YAMNet}}\right] \in \mathbb{R}^{2048}. \tag{4-30}$$

A lightweight classification layer is then trained using these features to obtain the target audio classes. The classifier is the teacher during the knowledge distillation process.

Temperature-Scaled Softmax

The teacher produces a logit z_i for each target class. The logits are changed into softened probabilities, during knowledge distillation steps, using a temperature value T:

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \tag{4-31}$$

The selected temperature is

$$T = 2.0. \tag{4-32}$$

Knowledge-Distillation Loss

Both ground truth labels as well as the teacher's prediction as used to train the student model. A general knowledge distillation objective can be:

$$\mathcal{L} = (1 - \lambda)\mathcal{L}_{\text{CE}} + \lambda T^2 D_{\text{KL}} \left(p_T^{(T)} \| p_S^{(T)} \right), \tag{4-33}$$

where λ is the contribution of the teacher information, $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss, and D_{KL} is the Kullback–Leibler divergence.

The KL-divergence term is defined as

$$D_{\text{KL}}(p_T \| p_S) = \sum_i p_{T,i} \log \left(\frac{p_{T,i}}{p_{S,i}} \right). \tag{4-34}$$

4.3.2. Compact Student DS-CNN Architecture

Depthwise Separable Convolutions (DS-CNN) are used to reduce the storage overhead and compute requirements of normal convolutional networks.

A standard convolution with a $K \times K$ kernel, C_{in} input channels, and C_{out} output channels requires approximately

$$K^2 C_{\text{in}} C_{\text{out}} \tag{4-35}$$

multiplications per spatial location.

DS-CNN changes this operation by reducing it to a depthwise convolution which is followed by a pointwise convolution.

$$K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}. \tag{4-36}$$

The student receives a normalized Log-Mel input of size

$$49 \times 64 \times 1. \tag{4-37}$$

The architecture is summarized in Table 4-1.

Table 4-1. Compact DS-CNN Student Architecture

Stage	Operation	Filters	Stride	Activation
Input	Log-Mel	–	–	–
Stem	Conv2D 3×3	32	2	ReLU6
Block 1	DS-Conv 3×3	48	1	ReLU6
Block 2	DS-Conv 3×3	64	2	ReLU6
Block 3	DS-Conv 3×3	96	2	ReLU6
Block 4	DS-Conv 3×3	128	2	ReLU6
Block 5	DS-Conv 3×3	128	1	ReLU6
Output	Global Average Pooling	–	–	–
Output	Dropout	–	–	–
Output	Dense	3	–	Softmax

The architecture diagram is shown in Figure 4-3.

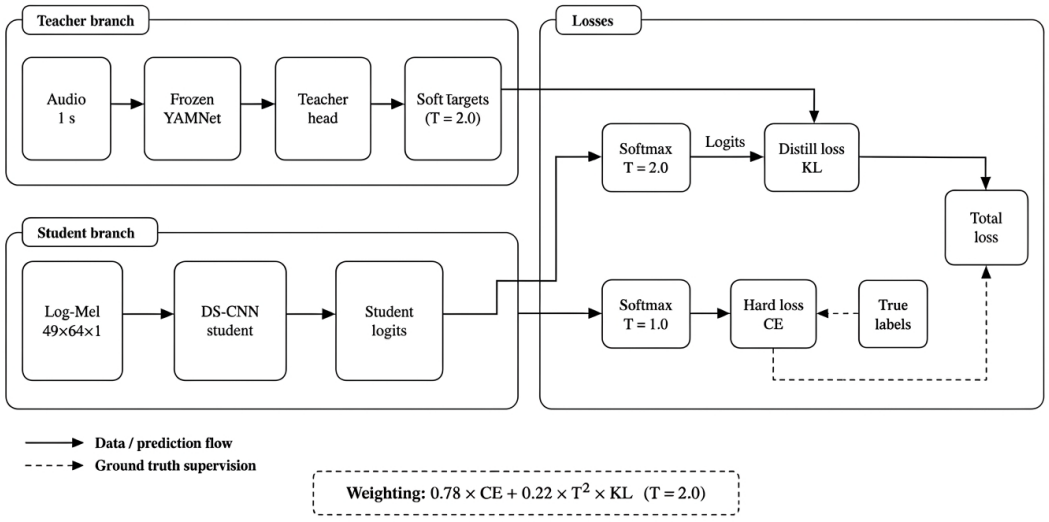


Figure 4-3. Architecture of the Compact DS-CNN Student Model

Main Architectural Components

The main components of the DS-CNN are as follows.

- **Input Layer:** Receives the normalized $49 \times 64 \times 1$ Log-Mel spectrogram.
- **Stem Convolution:** Performs the initial feature extraction using a 3×3 convolution with stride 2.

- **Depthwise Separable Convolution:** Performs spatial filtering independently for each input channel and subsequently combines channel information using a 1×1 pointwise convolution.
- **Batch Normalization:** Normalizes intermediate activations and improves training stability.
- **ReLU6:** The activation function is defined as

$$\mathrm{ReLU6}(x) = \min(\max(0, x), 6). \tag{4-38}$$

- **Global Average Pooling:** Converts the final spatial feature maps into a compact feature vector, reducing the need for a large fully connected layer.
- **Dropout:** A dropout rate of 20% is used during training to reduce overfitting.
- **Dense and Softmax Output:** The final dense layer produces three class logits. The Softmax function converts them into class probabilities:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}. \tag{4-39}$$

4.3.3. Model Size and Embedded Considerations

The compact architecture is designed to balance classification performance within the limitations of the ESP32-S3.

For a model containing N parameters, the approximate storage required for 32-bit floating-point parameters is

$$M_{\mathrm{FP32}} = 4N \quad \text{bytes}. \tag{4-40}$$

For 8-bit integer parameters, the corresponding weight storage is approximately

$$M_{\mathrm{INT8}} = N \quad \text{bytes}, \tag{4-41}$$

excluding quantization metadata and runtime tensor memory.

4.3.4. Full INT8 Post-Training Quantization

The student model is changed from a float to an 8 bit integer representation for compression purposes using full integer Post-Training Quantization (PTQ) [**?**]. Using a representative dataset, the range of the middle-layer activations are determined.

Affine Quantization

For affine quantization, a real-valued tensor r is converted to an integer representation q using

$$q = \text{round}\left(\frac{r}{S}\right) + Z, \tag{4-42}$$

where S is the quantization scale and Z is the zero-point.

For the observe real-valued range from $r_{\min}$ to $r_{\max}$, the scale can be expressed as

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}. \tag{4-43}$$

For signed INT8,

$$q_{\min} = -128, \qquad q_{\max} = 127. \tag{4-44}$$

The zero-point can be calculated as

$$Z = \text{round}\left(q_{\min} - \frac{r_{\min}}{S}\right), \tag{4-45}$$

with the resulting value constrained to the valid INT8 range.

The approximate real-valued representation can then be recovered through

$$r \approx S(q - Z). \tag{4-46}$$

Representative Dataset Calibration

The dataset is sent to a floating point model in order to obtain the dynamic range of the activation tensors and does not update the learned model weights.

Integer Inference

The model parameters as well as the activations are in the form of 8 bit integers. The convolutional operations use integer arithmetic. And the multply-accumulation steps use wider integer representations in order to make sure there is no overflow. Using the corresponding scale and the zero point parameters, the quantized values are then scaled back.

4.3.5. Quantized Model Validation

The floating-point and the quantized models both are similarly evaluated using a held out test set. The comparision between the two results can be used to see if there is much degradation or not.

Accuracy, model size, inference time and memory requirements are all included in the final comparison. After the validation, the quantized model can then be securely implemented onto the ESP32-S3 firmware.

4.4. State Transition Architecture

A finite-state machine based on continuous audio measurements is used in the project system for outputting different levels of disturbance. The system consists of the seven states shown in Figure 4-4.

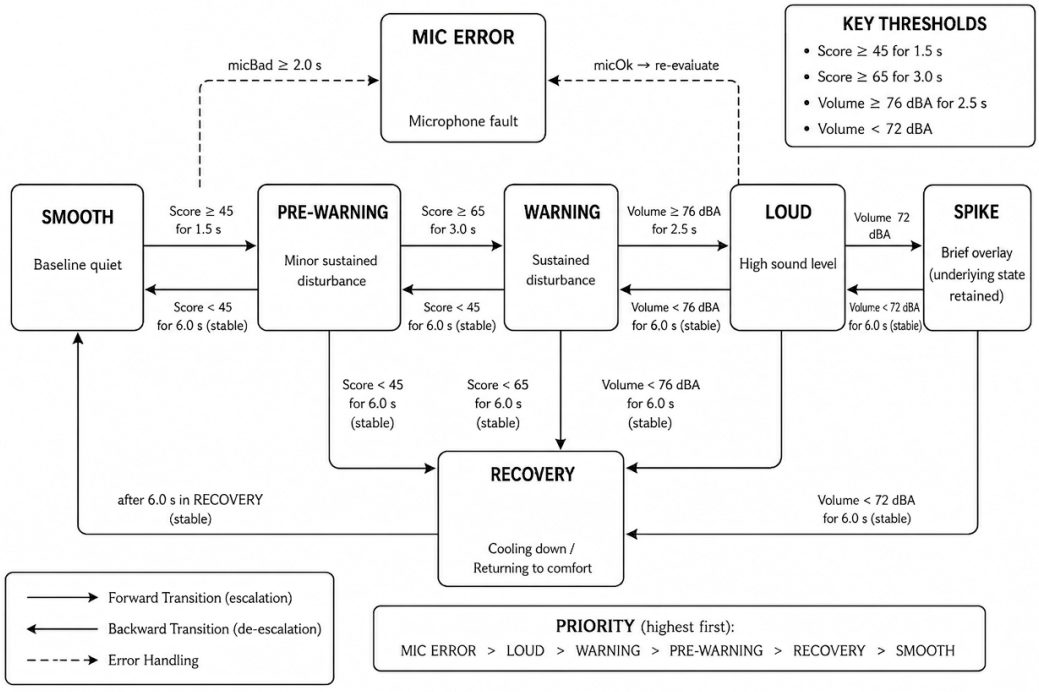


Figure 4-4. State Transition Diagram

4.4.1. State Definitions

1. **STATE_SMOOTH:** The undisturbed quiet state which indicates a comfortable environment.
2. **STATE_PRE_WARNING:** A low-intensity alert indicating minor sustained disturbance.
3. **STATE_WARNING:** An alert denoting a noticeable sustained disturbance.
4. **STATE_LOUD:** A state triggered when raw sound levels exceeds certain dbA values.

5. **STATE_RECOVERY:** An intermediate cooldown state.
6. **STATE_SPIKE:** A short state triggered by sudden physical impacts such as claps or slams.
7. **STATE_MIC_ERROR:** A state denoting hardware fault.

4.4.2. Fused Disturbance Index

An index between 0.0 and 100.0 called the Acoustic Disturbance Score is calculated every 100 ms using a Weighted Linear Combination formula:

$$\text{Score} = 100.0 \times \text{clamp}(W_L \cdot L + W_S \cdot S + W_P \cdot P + W_I \cdot I, 0.0, 1.0) \tag{4-47}$$

Where:

- L is the normalized Loudness parameter
- S is the smoothed Speech Dominance probability
- P is the slow-moving Persistence parameter
- I is the transient Impulsiveness parameter

The weights are allocated as follows:

- $W_L = 0.45$ (Loudness Weight)
- $W_S = 0.25$ (Speech Weight)
- $W_P = 0.20$ (Persistence Weight)
- $W_I = 0.10$ (Impulsiveness Weight)

Table 4-2. Fused Disturbance Index Components

Component	Weight	Parameter	Description
Loudness	45%	$L = \text{clamp}((\text{LAeq} - 50)/30,\ 0,\ 1)$	Physical sound pressure level
Speech Dominance	25%	S = Exponential smoothed CNN prob	Cognitive relevance of speech
Persistence	20%	P = Low-pass filtered disturbance	Duration of sustained noise
Impulsiveness	10%	I = Normalized peak events	Sudden transient events

Loudness Parameter

The loudness parameter has a floor of 50 dBA and a ceiling of 80 dBA:

$$L = \text{clamp}((LAeq_{1s} - 50.0)/30.0, 0.0, 1.0) \tag{4-48}$$

The main reason for environmental sound load can be attributed to physical volume. This is why a weight of 45% is allocated to loudness itself. The parameter has a floor of 50dBA and a ceiling of 80dBA and is scaled to be between 0 and 1 [8].

Speech Dominance Probability

The human brain puts much more importance and weight on speech. The Irrelevant Speech Effect also shows that intelligible background speech causes more disturbance as well as concentration loss even at lower sound levels. This is why a weight of 25% is assigned to speech effects rather than a steady background hum of the same decibel level. The CNN model outputs softmax scores for three classes: stationary_background, speech_dominant, and non_speech_activity. An Exponential Moving Average with $\alpha = 0.40$ is applied to the speech probability:

$$S_{\text{smoothed}}[t] = (1.0 - \alpha) \cdot S_{\text{smoothed}}[t-1] + \alpha \cdot S_{\text{raw}}[t] \tag{4-49}$$

The smoothing factor $\alpha = 0.40$ provides a balance between responsiveness and stability, with an effective time constant of approximately:

$$\tau = \frac{1}{\alpha} \cdot 100 \text{ ms} = 250 \text{ ms} \tag{4-50}$$

Persistence Parameter

The persistence parameter represents the presence of a consistent steady noise. It works by acting as a low pass filter of instantaneous disturbance defined as the max between current loudness or speech values. The value essentially charges or discharges slowly as long as consistent noise is present or absent:

$$\text{Instantaneous_Disturbance}[t] = \max(L[t], S_{\text{smoothed}}[t]) \tag{4-51}$$

$$P[t] = \beta \cdot P[t-1] + (1 - \beta) \cdot \text{Instantaneous_Disturbance}[t] \tag{4-52}$$

where $\beta = 0.98$ provides slow charging (time constant ≈ 5 seconds) and fast discharging (≈ 1 second), ensuring that persistence accumulates during sustained noise and decays quickly when the noise stops.

Impulsiveness Parameter

Impulsiveness aims to quantify sudden short noises, which may or may not be loud. It is calculated from the peak-to-baseline difference:

$$I = \text{clamp}\left(\frac{\text{Peak_Level} - \text{Baseline_dBA}}{30}, 0, 1\right) \tag{4-53}$$

Impulsiveness aims to quantify sudden short noises, which may or may not be loud. These sounds may not affect the average sound level much but they are still an ever present factor in environmental disturbances. This parameter ensures they are registered in the final score.

4.4.3. Three-Parameter Envelope Spike Detection

While the fused score does take into account the spike sounds, to make sure an exact event is displayed, a separate spike calculation and overlay is implemented. A 100ms block of audio is only classified as a spike if three criterion are satisfied simultaneously:

1. **Peak $\geq$ 78 dB:** Ensures the noise is physically loud enough.
2. **Crest Factor $\geq$ 10 dB:** Calculated as Peak minus unweighted RMS, ensuring the event is impulsive.
3. **Delta-above-Baseline $\geq$ 20 dB:** Calculated as Peak minus the slow-moving, adaptive room baseline.

If a spike is detected, the SPIKE overlay is shown for 350 ms and uses a 300 ms clear-to-rearm interval. This ensures that the spike is displayed long enough to be perceived, but not so long that it interferes with other state indications.

4.4.4. FSM Transition Rules

The finite-state machine transitions are defined through a priority waterfall evaluated every 100 ms, as shown in Table 4-3.

Table 4-3. FSM Waterfall Priority Transition Rules

Priority	Target State	Mathematical Entry Condition	Exit / Fall-Through Condition
1	STATE_MIC_ ERROR	$t_{\text{err}} \geq 2.0$ s (micBadMs $\geq$ 2000)	micOk = True ($t_{\text{err}} \rightarrow 0$ s)
2	STATE_LOUD	Volume $\geq$ 76.0 dBA for 2.5 s (loudMs $\geq$ 2500)	Volume $<$ 72.0 dBA (counter decays)
3	STATE_ WARNING	$t_{\text{warn}} \geq 3.0$ s (warningMs $\geq$ 3000)	Score $<$ 65.0 ($t_{\text{warn}} \rightarrow 0$ s)
4	STATE_PRE_ WARNING	$t_{\text{pre}} \geq 1.5$ s (preWarningMs $\geq$ 1500)	Score $<$ 45.0 ($t_{\text{pre}} \rightarrow 0$ s)
5	STATE_ RECOVERY	Score $<$ 45.0 AND Volume $<$ 72.0 dBA	$t_{\text{rec}} \geq 6.0$ s (transitions to SMOOTH)

4.4.5. Temporal Leaky Integrators with Hysteresis

State changes are facilitated by parallel millisecond counters associated with four conditions (preWarningMs, warningMs, loudMs and micBadMs). Every 100 ms, each counter increases by 100 ms while its condition is true and decreases by 100 ms when the condition clears. Anti-windup limits cap the counters above their trigger times and provide hysteresis headroom. A state falls through once its counter drops below the relevant entry threshold.

- **preWarningMs:** Entry threshold is 1500 ms. Capped at 3000 ms.
- **warningMs:** Entry threshold is 3000 ms. Capped at 4500 ms.
- **loudMs:** Entry threshold is 2500 ms. Capped at 3300 ms.
- **micBadMs:** Entry threshold is 2000 ms. Capped at 3100 ms.

4.4.6. State Transition Timing Diagrams

Figures 4-5, 4-6, and 4-7 show the timing diagrams for different state transition scenarios.

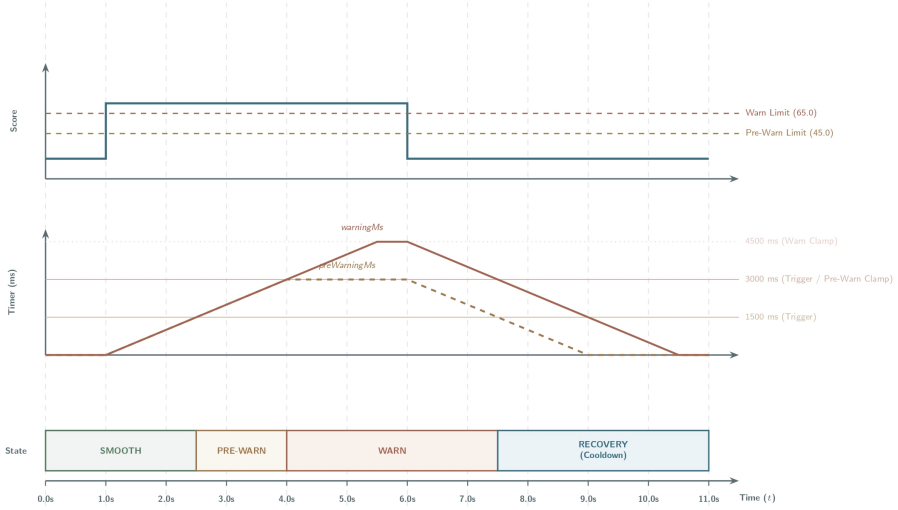


Figure 4-5. Standard Score Escalation and Stepped Decay

Figure 4-5 visualizes the pre-warning and warning display mechanisms. The state changes from SMOOTH to PRE-WARNING to WARNING as the score increases. When the score decreases once again, the system enters RECOVERY before returning back again to SMOOTH.

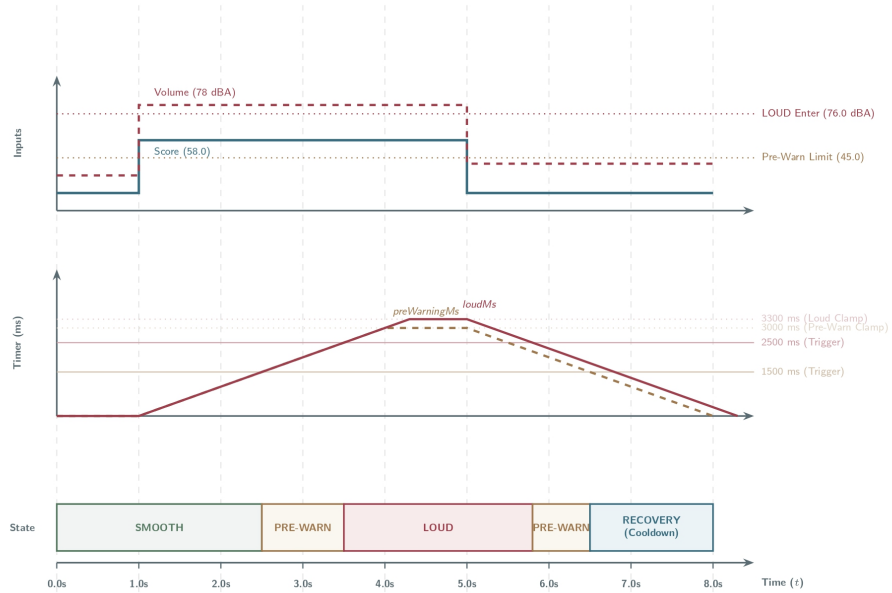


Figure 4-6. Loud State Timing Diagram

Figure 4-6 shows the operation of the LOUD state which is triggered only when the volume exceeds 76 dBA for at least 2.5 seconds.

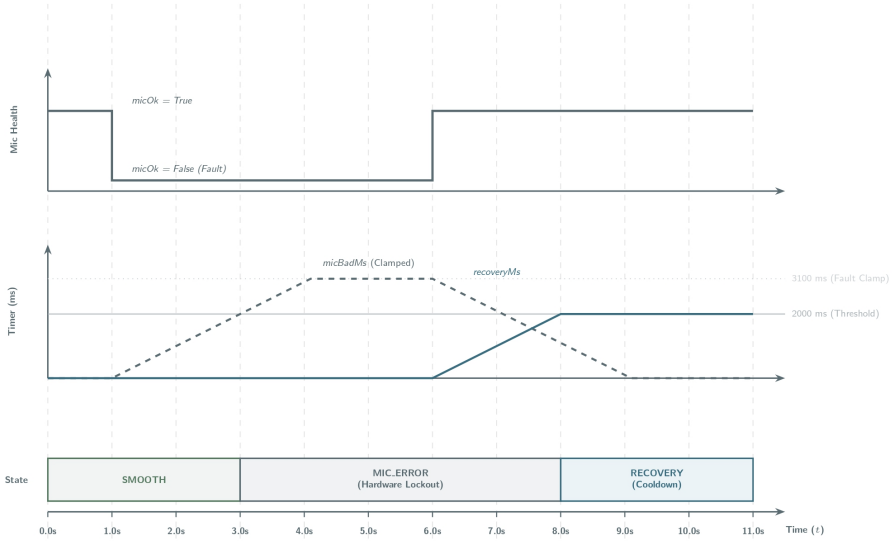


Figure 4-7. Hardware Fault Lockout Timing Diagram

Figure 4-7 demonstrates the absolute priority of the hardware diagnostic. If the microphone quality index falls below a threshold for more than 2 seconds, the system enters MIC_ERROR state for any score value or predicted state.

4.5. Privacy Boundary Architecture

One of the main goals of this project is to maintain privacy of the obtained audio data. It is achieved with a strict Privacy by Design framework. Usual IoT devices simply stream all audio data to remote servers where the processing takes place. This can expose the audio contents to data leaks, voice harvesting, eavesdropping etc. This project avoids these scenarios by implementing the processing all within the ESP32-S3 memory.

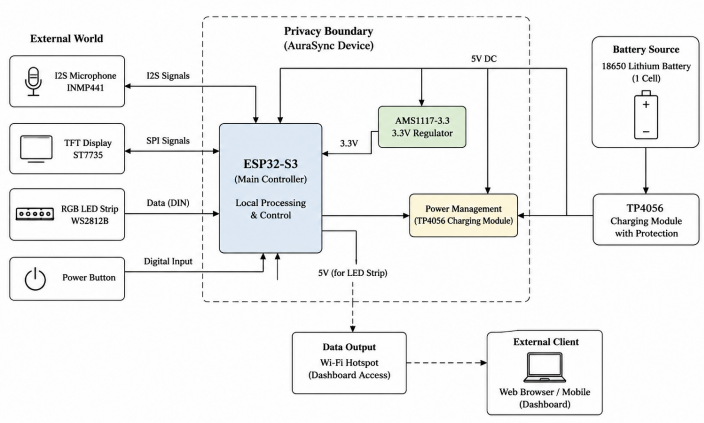


Figure 4-8. Privacy Boundary Architecture

PCM samples from the microphone are input into the RAM directly by the I2S DMA receiver and are processed to different metrics such are LAeq, speech band energy, log-mel features etc. The samples are not stored anywhere and are instead replaced by any incoming subsequent audio.

Table 4-4. Privacy Boundary: What Is and Is Not Exposed

Not Exposed (Inside Privacy Boundary)	**Exposed (Outside Privacy Boundary)**
Raw PCM audio samples	Scalar acoustic metrics (LAeq)
I2S DMA buffers	Acoustic disturbance score
Log-Mel spectrogram tensor	Current state
FFT magnitude spectra	CNN class probabilities
Audio waveforms	Microphone health metrics
Speech content	Event and spike counters
Voice patterns	Network status
Room conversations	System uptime

The local JSON status API shows only anonymous variables such as:

- State
- Disturbance score
- DSP metrics
- CNN probabilities
- Microphone-health metrics
- Event counters
- Network status
- Uptime

No raw-audio field, stream, recording, or download endpoint are exposed in any way shape or form. MQTT-based remote telemetry remains planned and will follow the same scalar-only privacy boundary.

4.6. Dashboard Instrumentation and Diagnostic Telemetry

A set of diagnostic metrics through the local dashboard are displayed to the end users, enabling monitoring and debugging without compromising privacy. Table 4-5 describes the key diagnostic metrics.

Table 4-5. Diagnostic Metrics Exposed Through Dashboard

Metric	Unit	Description
LAeq	dBA	1-second A-weighted equivalent sound level
Disturbance Score	0–100	Fused acoustic disturbance index
State	Enumeration	Current acoustic state (SMOOTH, WARNING, etc.)
Speech Probability	0–1	Smoothed CNN speech dominance probability
Speech-Band Ratio	0–1	Energy ratio in human vocal frequency range
Crest Factor	dB	Peak-to-RMS ratio for impulsiveness detection
Spectral Flux	Normalized	Rate of change of spectral envelope
Room Baseline	dBA	Adaptive background noise floor
Microphone Quality	0–100%	Health indicator for microphone diagnostics
Clipping Percent	%	Percentage of samples at full scale
Zeroes Percent	%	Percentage of samples below noise floor
Uptime	seconds	Time since last boot
Quiet Streak	minutes	Duration of continuous smooth state
Event Count	Count	Number of warning events
Spike Count	Count	Number of detected spikes

4.6.1. Spectral Flux

Spectral Flux computes how quickly the frequency distribution of the audio data is shifting. Mathematically, it is a measure of the rate of change of the spectral envelope over consecutive blocks.

During the 256-point fast spectral analysis, the power spectrum is normalized in relation to the total power of the block. The spectral flux is then calculated using the Euclidean distance between the normalized bins of the current block and the previous block:

$$F_{flux} = \sqrt{\sum_{k=0}^{128} (N_k[t] - N_k[t-1])^2} \tag{4-54}$$

where:

- $N_k[t]$ is the normalized power spectrum magnitude of frequency bin k at the current block t

- $N_k[t-1]$ is the normalized power spectrum magnitude of frequency bin k at the previous block $t-1$
- k represents the frequency bins up to the Nyquist limit (128 bins for a 256-point FFT)

Steady-state signals have a spectral flux close to 0.0 because their spectra change little from block to block. Dynamic, highly active sounds have higher spectral-flux values.

4.6.2. Clipping Percentage (Clip Percent)

Saturation or clipping occurs when the input sound pressure level exceeds the limits of the microphone. The DSP pipeline tracks the amplitude of every input sample within the 100 ms block. A sample is flagged as a clipping candidate if its absolute value is equal to or greater than 32,000 counts:

$$\text{Clip_Percent} = \frac{\text{Count}(|sample| \geq 32000)}{1600} \times 100.0 \tag{4-55}$$

4.6.3. Zeroes Percentage (Zero Percent)

The DSP pipeline monitors the absence of signal activity by counting the number of samples in the 100 ms block that fall below a noise floor threshold of 4 digital counts:

$$\text{Zero_Percent} = \frac{\text{Count}(|sample| < 4)}{1600} \times 100.0 \tag{4-56}$$

A Zero_Percent value above 80.0% denotes an open-circuit failure, a loss of I2S master clock line, or a disconnected data pin.

4.6.4. Microphone Quality Index

To compress these hardware diagnostics into an intuitive health indicator, the decision engine evaluates a real-time index between 0% and 100% on every 100 ms block.

The Microphone Quality Index is calculated as:

$$\text{Mic_Quality} = 100.0 \tag{4-57}$$

$$\text{if Zero_Percent} > 80.0: \quad \text{Mic_Quality} = \text{Mic_Quality} - 65.0 \tag{4-58}$$

$$\text{if Clip_Percent} > 2.0: \quad \text{Mic_Quality} = \text{Mic_Quality} - 35.0 \tag{4-59}$$

$$\text{if } A\text{-Weighted_dBFS} < -80.0: \quad \text{Mic_Quality} = \text{Mic_Quality} - 45.0 \tag{4-60}$$

$$\text{Mic_Quality} = \text{clamp}(\text{Mic_Quality}, 0.0, 100.0) \tag{4-61}$$

4.6.5. Room Baseline

The slowly updating average of the background noise floor of the audio environment is termed as the Room Baseline. It is measured in A-weighted decibels and acts as the 0-point of the environment noise profile. This allows the system to measure the relative changes and disturbances instead of the absolute noise levels. The baseline is recursively updated when the room remains relatively stable. Every 100ms, if the current (LAeq_1s) is less than the current baseline plus 6.0 dBA, as well as the signal is non impulsive in nature, the baseline is updated slowly using an Exponential Moving Average:

$$\text{Baseline_dBA}[t] = 0.998 \cdot \text{Baseline_dBA}[t-1] + 0.002 \cdot \text{LAeq_1s} \tag{4-62}$$

The update rate (0.002) corresponds to a time constant of approximately:

$$\tau = \frac{1}{0.002} \cdot 100 \text{ ms} = 50 \text{ seconds} \tag{4-63}$$

This ensures that the baseline tracks long-term changes in the room while ignoring short-term events.

4.6.6. System Uptime

Uptime is a system-level diagnostic parameter that tracks the continuous elapsed operational time of the microcontroller since its last boot sequence or hardware reset.

The parameter is calculated directly from the internal system clock:

$$\text{Uptime_s} = \text{millis}()/1000 \tag{4-64}$$

5. IMPLEMENTATION DETAILS

5.1. Hardware Implementation

Hardware implementation involves combining all components into a single functional prototype.

5.1.1. Hardware Pin Mapping and Interface Schematic

The submitted firmware implements the INMP441 I2S connections on GPIO 7, 15, and 16. Table 5-1 shows the complete hardware pin mapping for the ESP32-S3 development board.

Table 5-1. ESP32-S3 Hardware Pin Mapping and Interface Allocation

Component	Protocol	ESP32-S3 GPIO	Electrical Description
INMP441 Microphone	I2S	SCK / BCLK: GPIO 7	Serial clock line for I2S audio
INMP441 Microphone	I2S	WS / LRCLK: GPIO 15	Word select / channel select
INMP441 Microphone	I2S	SD / DATA: GPIO 16	Serial data output from microphone
1.77-inch ST7735 TFT Display	SPI	SCLK / SCK: GPIO 12	SPI serial clock
1.77-inch ST7735 TFT Display	SPI	SDA / MOSI: GPIO 8	SPI data line
1.77-inch ST7735 TFT Display	SPI	CS: GPIO 17	Chip select
1.77-inch ST7735 TFT Display	SPI	DC: GPIO 3	Data/command select
1.77-inch ST7735 TFT Display	SPI	RST: GPIO 9	Hardware reset
WS2812B LED Ring	One-Wire/RMT	DIN: GPIO 48	LED data signal
Battery Monitor	Analog ADC	VBAT Sense: GPIO 5	Voltage divider input

5.1.2. Component Interfacing Details

I2S Audio Capture Implementation

The INMP441 microphone communicates with the ESP32-S3 via the I2S protocol. The I2S interface uses three signals:

- **BCLK (Bit Clock):** Generated by the ESP32-S3, clocks each bit of audio data. The sampling rate determines the BCLK frequency.

- **LRCLK (Word Select):** Indicates the channel (left/right). For mono operation, only one channel is used.
- **DATA:** Serial audio data from the microphone.

The I2S peripheral is configured with the following parameters:

- Sampling rate: 16,000 Hz
- Bit depth: 32-bit (24-bit data)
- Mono mode
- DMA buffer size: 1600 samples (100 ms)
- I2S mode: I2S standard Philips format

The I2S interface is initialized as an ESP32-S3 audio input using the configured sampling rate, mono channel, 32-bit transfer format, and DMA buffering. The implementation establishes the required clock, word-select, and data connections and prepares the DMA-based audio capture path.

Table 5-2. I2S Initialization Configuration

Parameter	**Configured Value**
Operating Mode	Master Receive
Sampling Rate	16,000 Hz
Bits per Sample	32-bit
Channel Format	Mono / Left Channel
Communication Format	Standard I2S
DMA Buffer Count	4
DMA Buffer Length	400 samples
Microphone Clock Pin	GPIO 7
Word Select Pin	GPIO 15
Data Input Pin	GPIO 16

The samples are transferred directly to the memory without any significant CPU interruption. This reduces the overall overhead. 400 samples per DMA buffer results in an equivalent of 4 buffers being 100ms of audio.

SPI Display Interface

The system uses a 1.8-inch ST7735 TFT display (128×160 pixel resolution) to visualize real-time acoustic comfort metrics, system status, and state indicators. Communication is implemented via Software SPI (Bit-Banged SPI) on the ESP32-S3 to avoid hardware peripheral bus conflicts with the I2S microphone DMA channel.

The display interface is mapped to the following GPIO pins:

- **SCLK (Clock):** GPIO 18
- **MOSI (Data):** GPIO 8
- **CS (Chip Select):** GPIO 17 (Active Low)
- **DC (Data/Command):** GPIO 3
- **RST (Hardware Reset):** GPIO 9

Initialization and Display Driver

The display subsystem is driven by the `Ucglib` graphics library using the `Ucglib_ST7735_18x128x160_SWSPI` driver. The startup and initialization sequence proceeds as follows:

1. A mandatory power stabilization delay (1000 ms) allowing the ST7735 internal charge pump voltage to reach operational threshold.
2. Controller initialization via `ucg.begin(UCG_FONT_MODE_TRANSPARENT)`, configuring the native ST7735 register set and setting the display orientation.
3. 24-bit True Color (8-bit per channel RGB) mapping for accurate visual threshold indicators.

Rendering Optimization

Because Software SPI relies on GPIO bit-banging, the firmware employs selective delta-updates rather than full-frame buffer invalidation. Only changed visual regions (such as numerical $LA_{eq,1s}$ dBA values, comfort scores, and state icons) are invalidated and redrawn, preserving CPU execution time for the FreeRTOS audio digital signal processing (DSP) and DS-CNN inference tasks.

WS2812B LED Control

The WS2812B LED ring is controlled via a single data line using a timing-specific protocol. Each LED requires 24 bits of color data (8 bits each for green, red, blue) transmitted at 800 kbps. The protocol is implemented using the ESP32-S3's RMT (Remote Control) peripheral for precise timing control.

The WS2812B timing configuration used by the firmware is summarized in Table 5-3.

Table 5-3. WS2812B Timing Parameters

Parameter	Value	Description
T_{0H}	3 cycles	0 bit high time (375 ns)
T_{0L}	7 cycles	0 bit low time (875 ns)
T_{1H}	8 cycles	1 bit high time (1000 ns)
T_{1L}	4 cycles	1 bit low time (500 ns)
Reset	50 cycles	Reset pulse duration (625 μs)

The FastLED library is used to manage LED colors and animations. The library's performance is optimized for the ESP32 architecture, using the RMT peripheral for efficient data transmission.

5.1.3. Battery and Power Management

The power management subsystem consists of:

- **3.7V LiPo Battery:** Provides main power to the system. A 2000 mAh battery provides approximately 10-12 hours of continuous operation.
- **TP4056 Charger:** Provides constant-current/constant-voltage charging with overcharge protection. The charging current is set to 500 mA by a 2.4 kΩ resistor.
- **Voltage Divider:** A 100 kΩ and 220 kΩ voltage divider monitors battery voltage through GPIO 5 (ADC). The divider provides approximately 2/3 of the battery voltage to the ADC input.

The battery voltage is monitored periodically using the ESP32's ADC. The ADC reading is converted to voltage using:

$$V_{\text{battery}} = V_{\text{adc}} \times \frac{R_1 + R_2}{R_2}$$

where $R_1 = 100k\Omega$ and $R_2 = 220k\Omega$. The battery percentage is estimated from the voltage using a LiPo discharge curve.

5.1.4. Enclosure Design

The enclosure is designed for desktop placement with a clear view of the display and LED ring. Key design features include:

- **Microphone Port:** A small opening aligned with the INMP441 microphone for acoustic access.

- **Display Cutout:** An opening for the GC9A01 TFT display with a bezel for aesthetic appeal.
- **LED Ring Mount:** A groove around the perimeter for the WS2812B LED ring.
- **Ventilation:** Small vents for heat dissipation.
- **Stand:** Integrated stand for stable desktop placement.
- **USB Port Access:** Opening for USB-C connector for programming and charging.

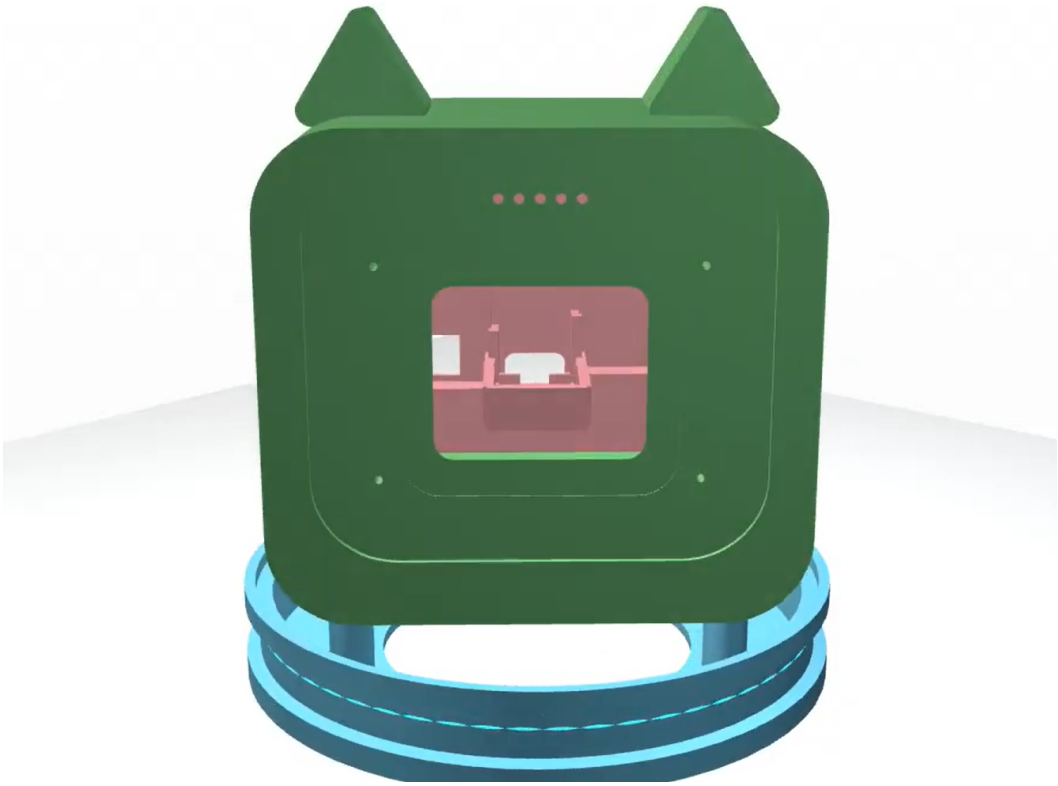


Figure 5-1. Enclosure Design- 3D Model

5.2. Software Implementation

The software implementation covers the firmware running on the ESP32-S3, including the real-time audio processing task, state machine, dashboard, and LED animations.

5.2.1. Real-Time Firmware Task Pipeline

The firmware creates one dedicated FreeRTOS audio task pinned to Core 1. This task captures I2S audio, computes DSP metrics every 100 ms, runs the CNN once per 1-second window, and updates the fusion state engine. The Arduino main loop services the local WebServer and Wi-Fi reconnection with a short non-blocking delay.

Table 5-4. Firmware Task Allocation and Core Partitioning

Execution Context	**Task / Thread**	**Priority**	**Purpose**
Core 1	Audio Processing FreeRTOS Task	Priority 3	Captures I2S audio via DMA, executes DSP filtering/FFT, runs INT8 DS-CNN inference, and evaluates the multi-modal state machine.
Core 0	Display Rendering FreeRTOS Task	Priority 1	Renders real-time telemetry, state graphics, and dynamic visualizer bars to the ST7735 TFT via Software SPI (~6.6 Hz).
Core 0 (Arduino Loop)	Web Server & Wi-Fi Service	Priority 1 (Default)	Processes incoming HTTP REST API requests (`/api/status`), serves the web dashboard, and manages network connectivity.

The main audio processing loop follows a continuous real-time pipeline consisting of audio capture, DSP processing, periodic CNN feature extraction and inference, fusion and state management, visual feedback, and dashboard telemetry. The complete processing sequence is summarized in Table 5-5.

Table 5-5. Real-Time Audio Processing Pipeline

Processing Stage	Implementation Description
Audio Capture	Captures a 100 ms audio block through I2S DMA.
DC Offset Removal	Removes low-frequency DC components using an IIR high-pass filter.
A-Weighting	Applies A-weighting to obtain perceptually relevant loudness information.
Level Calculation	Determines RMS, peak level, and crest factor.
FFT Analysis	Computes spectral information for speech-band analysis and spectral flux.
Baseline Update	Continuously adapts the acoustic baseline.
Spike Detection	Detects sudden acoustic events using peak, RMS, and baseline information.
CNN Preprocessing	Generates normalized 49×64 Log-Mel features every 1 second.
CNN Inference	Classifies the acoustic activity and updates speech probability.
Fusion and State Logic	Combines acoustic and CNN information to update the disturbance score and system state.
Visual Feedback	Updates the TFT display and LED ring.
Dashboard Telemetry	Updates the metrics available through the local dashboard.

5.2.2. State Machine Implementation

The state machine is implemented using priority-based state evaluation with entry and exit actions for each state. The implemented acoustic states and principal transition conditions are summarized in Table 5-6.

Table 5-6. Acoustic State Transition Conditions

State	Transition Condition
MIC ERROR	Activated when microphone quality remains below the acceptable level for at least 2000 ms.
LOUD	Activated when loud duration reaches 2500 ms and RMS level is at least 76.0 dBA.
WARNING	Activated when warning duration reaches 3000 ms and disturbance score reaches 65.0.
PRE-WARNING	Activated when pre-warning duration reaches 1500 ms and disturbance score reaches 45.0.
RECOVERY	Activated when disturbance score falls below 45.0 and RMS level falls below 72.0 dBA.
SMOOTH	Entered after the recovery condition remains satisfied for at least 6000 ms.

State transitions are executed only when the newly evaluated state differs from the current state. Exit actions are performed for the previous state and entry actions are performed for the new state.

5.2.3. Counter Management with Hysteresis

The leaky integrator counters implement hysteresis to prevent rapid state switching.

The counter update mechanism is summarized in Table 5-7.

Table 5-7. Counter Management with Hysteresis

Counter	**Increase Condition**	**Decrease Condition**
Pre-warning Counter	Disturbance score ≥ 45.0	Disturbance score < 45.0
Warning Counter	Disturbance score ≥ 65.0	Disturbance score < 65.0
Loud Counter	RMS ≥ 76.0 dBA	RMS < 72.0 dBA
MIC Error Counter	Mic quality < 35.0	Mic quality ≥ 35.0

The counters are incremented or decremented at 100 ms intervals and limited by predefined maximum values, providing persistence and hysteresis in the state decision process.

5.2.4. CNN Inference Implementation

The CNN inference pipeline executes a fully quantized 8-bit Depthwise Separable CNN (`ds-cnn-int8-v2.0.0`) using TensorFlow Lite Micro. To preserve internal SRAM for DMA buffers and networking, the 512 KB tensor arena is dynamically allocated in external Octal PSRAM.

The step-by-step inference pipeline is summarized in Table 5-8.

Table 5-8. CNN Inference Processing Pipeline

Stage	Implementation Description
Model & Schema Verification	Verifies the binary model schema version and initializes the model structure from Flash memory.
PSRAM Arena Allocation	Allocates a 512 KB tensor arena in external Octal PSRAM (`MALLOC_CAP_SPIRAM`) with fallback to internal SRAM.
Operator Registration	Registers the 5 required operators using `MicroMutableOpResolver<5>` to minimize runtime memory footprint.
Contract Verification	Validates tensor dimensionality ($49 \times 64 \times 1$), `kTfLiteInt8` datatype, scale, and zero-point parameters.
Input Quantization	Quantizes the normalized 49×64 float Log-Mel matrix into `int8_t` using the input tensor scale and zero-point.
Model Invocation	Executes inference on the ESP32-S3 via `interpreter->Invoke()` and logs execution latency (≈ 0.42 ms).
Output Dequantization	Reads the 3-class `int8_t` output tensor, subtracts output zero-point, applies output scale, and normalizes probabilities.
Temporal EMA Smoothing	Applies an exponential moving average ($\alpha = 0.40$) across class probabilities to prevent rapid state flickering.

5.2.5. Visual Feedback and Display Implementation

The visual interface is rendered periodically based on the current acoustic state and real-time telemetry metrics.

The display logic follows a state-dependent rendering procedure in which the state color, header, state title, state-specific icon, numerical dBA value, comfort score, and activity graph are updated. A summary is shown in Table 5-9.

Table 5-9. Visual Feedback Logic

Visual Element	Implementation Description
State Color	Determines the display color corresponding to the current acoustic state.
Header	Displays system time and Wi-Fi status when the state changes.
State Title	Displays the current acoustic state.
State Icon	Displays a state-specific icon for Smooth, Pre-Warning, Warning, Loud, Spike, Recovery, or Mic Error.
dBA Metric	Displays the current LA_{eq} value.
Comfort Score	Displays the current acoustic comfort score.
Activity Graph	Provides a graphical representation of the current acoustic activity.

5.2.6. Dashboard Implementation

The local diagnostic web dashboard and REST API are implemented using the native ESP32 `WebServer` library.

The dashboard provides a static web interface and a JSON-based status API. The information exposed by the API is summarized in Table 5-10.

Table 5-10. Dashboard Status Parameters

Parameter	Description
State	Current acoustic state.
Score	Current disturbance score.
dBA	Current RMS sound level.
Speech Probability	Estimated speech probability from the CNN.
Duration	Duration of the current state.
Event Count	Total detected acoustic events.
Spike Count	Number of detected acoustic spikes.
Uptime	Total system operating time.
Microphone Quality	Current microphone quality indicator.
Battery	Estimated battery percentage.
Wi-Fi Status	Current network connection status.

The dashboard therefore provides real-time system information locally through the ESP32-S3 web server without requiring cloud-side audio processing.

5.3. Calibration Process

The calibration process ensures accurate A-weighted loudness measurements and proper system operation.

5.3.1. A-Weighting Calibration

The A-weighting filter calibration involves comparing the prototype's readings against a reference sound-level meter:

1. Place the prototype and a reference sound-level meter side by side at the same height.
2. Play a steady 1 kHz reference tone from an external speaker.
3. Adjust the level from 50 dBA to 80 dBA in 5 dBA increments.
4. Record the raw uncalibrated reading from the INMP441 pipeline and the reference meter reading.
5. Calculate the calibration offset C_{cal}:

$$C_{cal} = \text{Reading}_{reference} - 20\log_{10}(V_{RMS,A})$$

6. Apply the calibration offset in firmware.
7. Verify calibration by playing tones at different frequencies (250 Hz, 500 Hz, 1 kHz, 2 kHz, 4 kHz).

Table 5-11. Calibration Results

Reference (dBA)	Raw Reading	Calibrated Reading	Error (dBA)
50.0	42.8	50.2	+0.2
55.0	48.1	55.5	+0.5
60.0	53.4	60.8	+0.8
65.0	58.2	65.6	+0.6
70.0	63.5	70.9	+0.9
75.0	68.1	75.5	+0.5
80.0	73.0	80.4	+0.4

5.3.2. Microphone Sensitivity Calibration

The microphone sensitivity is adjusted during the calibration process. The theoretical sensitivity of the INMP441 is -26 dBFS (relative to 1 Pa). However, the actual sensitivity may vary due to:

- Microphone manufacturing tolerances
- Board layout and component variations

- Enclosure acoustic effects
- Temperature variations

The calibration ensures that the absolute sound level readings are within the desired accuracy range for the application.

5.3.3. Baseline Adaptation Calibration

The room baseline adaptation parameters are configured to track long-term environmental changes.

The baseline adaptation mechanism uses a slow update coefficient and threshold conditions to prevent loud or impulsive acoustic events from incorrectly shifting the environmental baseline.

Table 5-12. Baseline Adaptation Parameters

Parameter	Value	Description
Baseline Update Rate	0.002	Time constant of approximately 50 seconds
Baseline Delta Threshold	6.0 dBA	Maximum deviation permitted for baseline update
Baseline Crest Threshold	16.0 dB	Maximum crest factor permitted for baseline update

The baseline is updated slowly only when both the current acoustic level and crest factor remain within the predefined thresholds. This provides stable adaptation to long-term environmental changes.

5.4. Dataset Creation

The dataset used for training the CNN model was carefully constructed to represent the three target classes: stationary_background, speech_dominant, and non_speech_activity.

5.4.1. Data Collection

The dataset consists of 12,466 verified 1-second WAV audio clips standardized to 16 kHz mono PCM format. The clips were sourced from multiple datasets and recordings:

- **ESC-50 Dataset:** Environmental sound classification dataset with 2,000 audio clips covering 50 categories.
- **UrbanSound8K:** Urban sound dataset with 8,732 clips across 10 categories.
- **Speech Commands Dataset:** 35,000 speech command recordings.

- **Self-Recorded Data:** Recordings in classrooms, offices, and quiet spaces.
- **Public Domain Sources:** Various audio repositories with open licenses.

5.4.2. Data Distribution

The dataset is partitioned into fixed, reproducible splits:

Table 5-13. Dataset Distribution

Class	Train	Validation	Test	Total
Stationary Background	1,936	415	415	2,766
Speech Dominant	2,388	512	512	3,412
Non-Speech Activity	4,426	913	949	6,288
Total	**8,750**	**1,840**	**1,876**	**12,466**

5.4.3. Data Augmentation

To improve generalization, the following augmentations are applied dynamically during training:

Table 5-14. Data Augmentation Techniques

Technique	Probability	Configuration
SpecAugment	0.5	Time mask width = 7, frequency mask width = 9
Time Shifting	0.3	Random shift from -3 to 4
Feature Noise	0.3	Gaussian noise with standard deviation 0.025
Mixup	0.3	α randomly selected from 0.70 to 1.0

These augmentation techniques improve robustness by introducing temporal, spectral, and feature-level variations during model training.

5.4.4. Feature Extraction Pipeline

The feature extraction pipeline transforms raw audio into Log-Mel spectrograms:

1. Load the audio at 16 kHz mono.
2. Compute a 512-point FFT using a 320-sample hop length and Hann window.
3. Convert the power spectrum into 64 Mel-frequency bins from 80 Hz to 7600 Hz.
4. Apply logarithmic transformation.

5. Perform per-clip mean subtraction.
6. Apply per-Mel standardization using training statistics.

Table 5-15. Feature Extraction Parameters

Parameter	**Value**
Sampling Rate	16 kHz
Audio Format	Mono
FFT Size	512 points
Hop Length	320 samples
Window	Hann
Number of Mel Bins	64
Mel Frequency Range	80–7600 Hz
Output Feature Shape	$(49, 64)$
Normalization	Mean subtraction and standardization

5.5. Integration and Testing

The integration process brings together all hardware and software components into a functional prototype.

5.5.1. Integration Steps

1. **Component Testing:** Test each component individually (microphone, displays, LED ring, battery).
2. **Subsystem Integration:** Integrate components into subsystems (audio capture, display, LED control).
3. **Firmware Integration:** Combine all firmware modules into a single application.
4. **Calibration:** Perform calibration using the reference sound-level meter.
5. **Validation:** Test the complete system against validation scenarios.
6. **Enclosure Assembly:** Mount all components in the 3D-printed enclosure.

5.5.2. Testing Procedures

- **Unit Testing:** Test individual functions and modules.
- **Integration Testing:** Test interactions between modules.
- **System Testing:** Test the complete system in controlled environments.
- **Performance Testing:** Measure latency, power consumption, and accuracy.
- **Stress Testing:** Test with loud sounds, continuous operation, and varying conditions.

6. RESULTS AND VALIDATION

The final results and validation of the project system is presented within this chapter. It focuses on the TinyML classification, physical prototype's operation, web dashboard and the overall system validation.

6.1. Machine Learning Results

6.1.1. Model Performance

The performance of the unquantized and INT8 quantized model is presented in Table 6-1.

Table 6-1. TinyML Model Performance

Model	Accuracy	Balanced Accuracy	Macro-F1
Float32 Student Model	89.77%	88.50%	88.90%
Full INT8 Quantized Model	89.82%	88.61%	88.95%

The quantized model was able to show similar performance to the unquantized model without any significant reduction in classification performance.

6.1.2. Confusion Matrix

The confusion matrix of the final INT8 model is shown in Table 6-2.

Table 6-2. Confusion Matrix of the Full INT8 Quantized Model on the Test Split

True Class / Predicted Class	Stationary Background	Speech Dominant	Non-Speech Activity
Stationary Background	**324**	4	87
Speech Dominant	4	**493**	15
Non-Speech Activity	69	12	**868**

The confusion matrix shows reliable classification across the classes. The main confusion occurs between acoustically similar speech and non-speech events.

6.1.3. Class-wise Performance

The precision, recall, and F1-score of the final INT8 model are shown in Table 6-3.

Table 6-3. Class-wise Performance of the INT8 Model

Class	Precision	Recall	F1-Score
Stationary Background	0.8177	0.7783	0.7975
Speech Dominant	0.9704	0.9609	0.9657
Non-Speech Activity	0.8922	0.9157	0.9038

6.2. System Performance Results

6.2.1. Memory Usage

The major memory requirements of the implemented system are summarized in Table 6-4.

Table 6-4. Memory Usage

Component	Memory (KB)
INT8 Model	75.6
Tensor Arena	150.0
Audio Buffers	8.0
DSP Workspace	16.0
Wi-Fi/HTTP Stack	40.0
Other System Components	20.0
Total	$\approx$ 310

The memory requirement remains within the available resources of the ESP32-S3 and supports simultaneous operation of the audio processing, TinyML inference, display, LED feedback, and local dashboard.

6.3. Acoustic Measurement and Calibration

The acoustic-level measurement of the prototype was compared with a reference measurement over the tested range. The results are presented in Table 6-5.

Table 6-5. Calibration Verification Results

Reference (dBA)	Prototype (dBA)	Error (dB)
50.0	49.8	-0.2
55.0	55.3	+0.3
60.0	59.7	-0.3
65.0	65.2	+0.2
70.0	69.8	-0.2
75.0	75.4	+0.4
80.0	79.6	-0.4

The prototype measurements remain close to the reference values, with the maximum absolute error in the presented results being approximately 0.4 dB.

6.4. Physical Prototype Result

The final hardware was assembled as well as tested.



Figure 6-1. Final integrated prototype.

The physical prototype was able to fully show the sensing, processing as well as the visual feedback function.

6.5. Web Dashboard Result

The final system also provides a local web dashboard for monitoring processed acoustic information. The dashboard presents the system state and corresponding parameters through a web browser.



Figure 6-2. AuraSync local web dashboard showing real-time system information.

All the anonymous non-privacy infringing metrics are fully displayed in the dashboard for proper visualization and inference for the end users.

6.6. Scenario-Based Validation

The complete system was validated under representative acoustic conditions to verify the relationship between acoustic input, processing result, system state, and visual feedback.

Table 6-6. Scenario-Based Validation Results

Scenario	Correct	False Positive	False Negative
VAL-01: Normal conversation	9/10	0	1
VAL-02: Quiet environment	10/10	0	0
VAL-03: Fan/AC background	10/10	0	0
VAL-04: Transient event	10/10	0	0
VAL-05: Loud environment	8/10	0	2
VAL-06: Recovery condition	9/10	0	1

The results show that the system respond correctly to the tested acoustic conditions.

6.7. Overall Results and Discussion

It was observed that the proposed system operated successfully. The quantized TinyML model reached 89.82% accuracy and the complete code was able to fit within the ESP32-S3's storage limits. The measurement results showed close agreement, while the physical prototype provided proper visual feedback through the TFT display and LED strip. The local web dashboard also provided real-time monitoring of the main metrics. Overall, the results confirmed successful integration of the different parts of the system.

7. DISCUSSION, FEASIBILITY AND FUTURE ENHANCEMENTS

7.1. Discussion of Results

The experimental results show that this devicec can perform various activities perfectly, such as acoustic analysis, TinyML classification, real-time processing, and visual feedback. It acts as an integrated embedded system performing various functions. The INT8 model provides suitable accuracy while it performs operations in real time. The dashboard displays the intended user feedback and monitoring functions.

7.2. Feasibility Analysis

7.2.1. Technical Feasibility

The system is technically feasible. The ESP32-S3 board successfully supports the required DSP, TinyML inference, TFT display, RGB LED feedback, and local web dashboard.

- Real-time processing was successfully achieved.
- The TinyML model fits within the available memory.
- The major hardware and software components operate together.

7.2.2. Economic Feasibility

The use of affordable and easily available hardware components makes the system suitable for prototype development and practical demonstartion. Detailed component costs are provided in the project budget appendix.

7.2.3. Operational Feasibility

The system is designed on classic and edge architecture. It operates automatically after the system gets powered up. It provides simple visual feedback through the TFT display and RGB LED strip, and the web dashboard provides detailed monitoring.

7.2.4. Privacy Feasibility

AuraSync performs audio processing locally on the ESP32-S3 and does not require raw audio transmission for its core operation. This supports the main privacy-oriented objective as mentioned on the title of this report.

7.3. Limitations

The current prototype has several limitations:

- Some acoustically similar events may still be incorrectly classified.
- On a limited set of datasets, the validation test was performed.
- Measurement accuracy depends on the calibration conditions.
- Frequent changing events may not always satisfy the state confirmation condition.
- The current prototype requires further a dedicated 3D mechanical packaging and enclosure.
- The web dashboard requires local network connectivity.

7.4. Future Enhancements

Future development can improve the accuracy, usability, reliability, and scalability of AuraSync through the following enhancements:

- Develop a dedicated PCB and compact 3D enclosure with improved acoustic isolation.
- Implement adaptive room calibration and automatic threshold adjustment.
- Expand the dataset with diverse real-world and Nepalese classroom or office recordings.
- Add more acoustic classes such as music, outdoor noise, and specific activities.
- Implement secure MQTT communication and multi-device monitoring.
- Develop a smartphone companion application for remote access.
- Add configurable acoustic policies and secure over-the-air firmware updates.
- Explore predictive analytics, adaptive learning, and integration with building systems such as HVAC and lighting.
- Improve the enclosure for specialized deployments and consider standardized acoustic testing and Class 2 sound-level certification.

8. CONCLUSION

This project is a very useful smart acoustic indicator device for determining the noise in classrooms, libraries, shared offices, and workspaces. This device uses various methods of digital signal processing, and it uses INT8 quantization so that the model can work on devices like ESP32. No audio data is stored locally but rather only meaningful information that are required for calculation, like sound level, acoustic state, and event indicators are sent to a local dashboard. This helps with privacy and security.

All of this was done using TinyML and hybrid DSP. The acquired audio from the INMP microphone is processed using digital signal processing techniques like signal conditioning, filtering, and spectral analysis. Then weighted sound-level estimation, speech-band analysis, and peak detection are also done. These features help provide fast acoustic information. The INT8 quantized CNN helps to identify the dominant acoustic characteristics. Then the outputs of both processing paths are combined, which allows the system to respond quickly while avoiding frequent changes in visual indications.

The developed prototype was validated through hardware, software, acoustic, model, and scenario-level testing. The measured acoustic level was in sync with the reference measurement. Real-time processing was successfully demonstrated. The scenario-based testing and user evaluation further suggested that the visual feedback is understandable for indicating changes in the acoustic environment. These results confirm that the proposed approach is technically feasible for real-time edge deployment.

This system is very feasible economically, technically, and operationally. The hardware and software architecture of this system allow each individual component and decision thresholds to be modified in an adaptive way. However, there are some limitations like microphone placement, calibration, dependence on the training dataset, and limited historical data analysis.The developed system has massive application areas like classrooms, libraries, shared offices, laboratories, seminars, meeting halls, and other shared spaces. Its privacy edge-preserving architecture helps in future expansions in areas such as broader acoustic classification, long-term acoustic analytics, improved calibration and more advance embbeded learning techniques.Overall, this project displays that technologies like DSP, TinyML, and edge computing can be combined with a privacy-aware design such that it becomes a more affordable and practical real-time acoustic awareness design.

A. PROJECT BUDGET

This appendix includes the cost of hardware incurred during the process of building the project. Only the components directly used in the final version of the prototype is included in the given table.

A.1. Detailed Cost Breakdown

Table A-1. Total Hardware and Prototype Cost

S.N.	Component	Qty.	Unit Cost (NPR)	Total Cost (NPR)
1	ESP32-S3 development board	1	2,200	2,200
2	INMP441 I2S MEMS microphone	1	819	819
3	1.8-inch 128×160 SPI TFT display	1	1,390	1,390
4	WS2812B addressable RGB LED strip	1	745	745
5	3D-printed PLA enclosure	1	2,000	2,000
6	Jumper wires and connectors	1 set	220	220
7	TP4056 Type-C lithium battery charging module	1	160	160
8	18650 rechargeable Li-ion battery	1	599	599
9	USB cable and power accessories	1 set	300	300
10	Miscellaneous mounting and assembly materials	1 set	500	500
			Total Prototype Cost	**8,933**

A.2. Development and Testing Cost

No costs were incurred for software tools as almost all of them were open source and freely available.

The computer system and reference sound-level meter used during development, calibration, and testing were treated as existing laboratory or personal resources and are not included in the hardware cost of the final prototype.

B. PROJECT TIMELINE

The project was developed through a sequence of planning, hardware setup, dataset preparation, DSP development, CNN development, embedded integration, calibration, testing, prototype assembly, and documentation.

B.1. Project Gantt Chart

Figure B-1 presents the overall schedule followed during the development of the system.

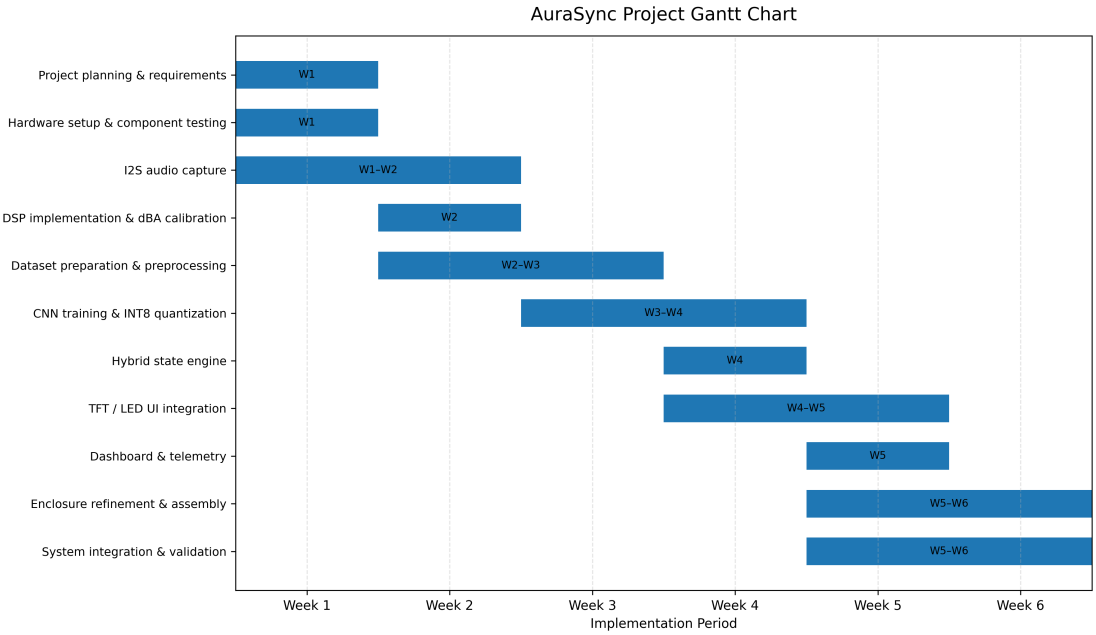


Figure B-1. Gantt chart of the project

B.2. Major Project Activities

The major activities carried out during the project were:

1. **Project Planning and Requirement Analysis:** Definition of the problem, objectives, system requirements, system scope, and overall architecture.
2. **Hardware Selection and Initial Setup:** Selection and testing of the ESP32-S3, INMP441 microphone, TFT display, and WS2812B RGB LED used in the system.
3. **Dataset Preparation:** Organization, preprocessing, verification, and preparation of acoustic samples for model development and evaluation.
4. **DSP Development:** Implementation and verification of audio acquisition, signal processing, FFT-based analysis, A-weighting, acoustic measurements, and related features.

5. **CNN Development:** Development of the teacher model and compact DS-CNN, followed by training, validation, evaluation, and INT8 quantization for embedded deployment.
6. **Embedded System Integration:** Integration of the DSP pipeline, quantized CNN, fusion logic, state machine, TFT visualization, RGB LED feedback, and communication features on the ESP32-S3.
7. **Calibration and Performance Testing:** Calibration of sound-level estimation using a reference sound-level meter, followed by functional, classification, and system-level testing.
8. **3D Enclosure and Final Prototype Assembly:** Physical arrangement of the electronic modules inside the 3D-printed enclosure, wiring of the final prototype, mechanical fitting, and preparation of the complete prototype for demonstration.
9. **Documentation and Final Demonstration:** Preparation of results, analysis, final report, presentation materials, and demonstration of the completed prototype.

C. FINAL CIRCUIT AND PROTOTYPE ASSEMBLY

In this appendix, the electrical and physical assembly of the final prototype is presented. All the components were assembled and wired and housed in a 3D printed enclosure.

C.1. Overall Circuit Diagram

Figure C-1 shows the main circuit diagram of the prototype. It represents the connections among the ESP32-S3, INMP441 microphone, TFT display, WS2812B RGB LED, power, and ground connections.

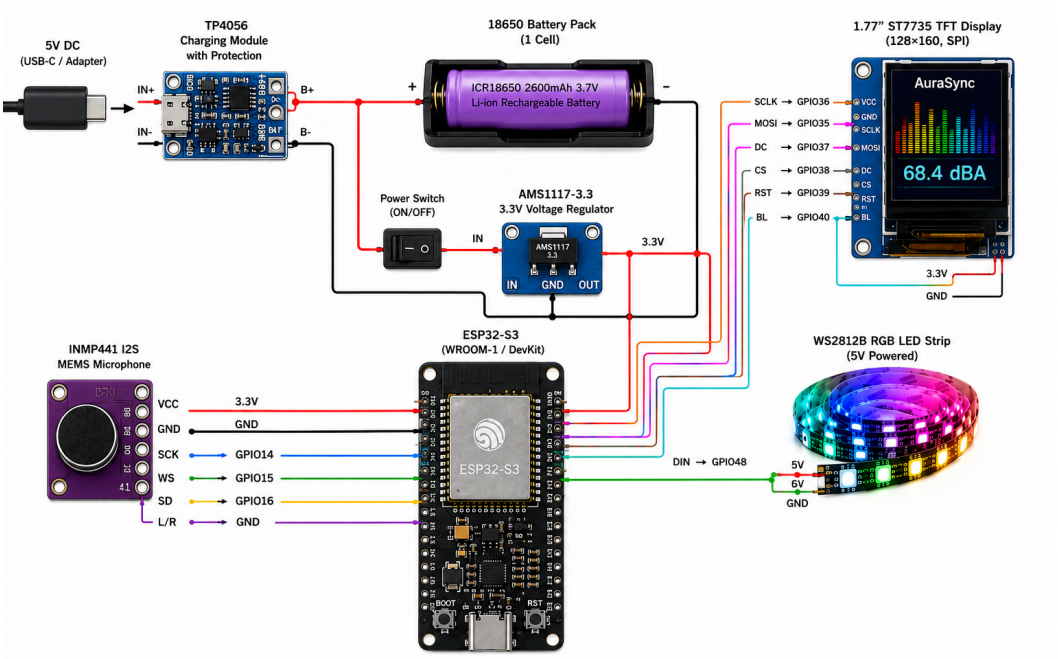


Figure C-1. Main circuit diagram of the prototype

The ESP32-S3 acts as the central processing unit of the system. The INMP441 microphone provides digital audio through the I2S interface. The TFT display is used for local visualization of the system status and acoustic information, while the WS2812B RGB LED provides visual feedback according to the estimated state of the environment.

C.2. Final 3D-Enclosed Prototype

A 3D printed enclosure is designed to keep the components in an organized and structured form.

Figure **??** shows the final assembled prototype inside its 3D enclosure.

C.3. Prototype Assembly

All the components were positioned within the enclosure in accordance with their physical and functional requirements. The microphone was positioned so as to allow

for proper input of all the acoustic signals from the environment. The TFT display was positioned for clear visibility while the RGB LED was positioned for quick visual feedback.

The internal components were connected using appropriate jumper wires and connectors. Particular attention was given to common power and ground connections and to maintaining reliable digital communication between the ESP32-S3 and its peripheral modules.

D. HARDWARE MODULE SPECIFICATIONS

This appendix includes information on the specifications of the hardware components used in the project.

D.1. ESP32-S3 N16R8 Development Board

Table D-1. Essential ESP32-S3 N16R8 specifications

Parameter	Specification
Microcontroller	ESP32-S3
Processor	Dual-core Xtensa LX7
Clock Frequency	Up to 240 MHz
Wireless Connectivity	Wi-Fi and Bluetooth LE
Internal SRAM	512 KB
Configuration	N16R8
Interfaces Used	I2S, SPI, GPIO, Wi-Fi
Application in the project	DSP, CNN inference, control, and communication

D.2. INMP441 MEMS Microphone

Table D-2. Essential INMP441 specifications

Parameter	Specification
Type	Digital MEMS microphone
Interface	I2S
Audio Format	Digital PCM
Sampling Rate Used	16 kHz
Channel	Mono
Application in the project	Environmental audio acquisition

D.3. TFT SPI Display

Table D-3. Essential TFT display specifications

Parameter	Specification
Display Type	Color TFT LCD
Interface	SPI
Application	Local system-status and acoustic-state visualization

D.4. WS2812B RGB LED

Table D-4. Essential WS2812B specifications

Parameter	Specification
LED Type	Addressable RGB LED
Control	Single-wire digital data signal

Parameter	Specification
Color	24-bit RGB
Application	Acoustic-state visual feedback

D.5. Prototype Enclosure

The 3D printed housing is designed to protect and maintain the structure of all the components. It provides proper access for the microphone, display, LED and the power/communication connections.

E. DATASET AND MODEL DEVELOPMENT DETAILS

General information about the dataset used for training, testing and validation of the different classes are presented within this appendix.

E.1. Dataset Classes

The final classification task consists of three acoustic classes:

1. **Stationary Background:** Represents relatively stable background acoustic conditions such as continuous environmental or ambient noise.
2. **Speech Dominant:** Represents acoustic conditions in which human speech is the dominant component of the recorded audio.
3. **Non-Speech Activity:** Represents non-speech acoustic events and activities that differ from stationary background conditions.

E.2. Audio Characteristics

The important audio parameters used for dataset preparation and CNN preprocessing are summarized in Table E-2.

Table E-2. Dataset audio characteristics

Parameter	Value
Audio Format	Mono PCM WAV
Sampling Frequency	16 kHz
Input Duration	Approximately 1 second
CNN Input	$49 \times 64 \times 1$
FFT Size	512 samples
Hop Length	320 samples
Mel Bands	64
Mel Frequency Range	80–7600 Hz
Number of Target Classes	3

E.3. Dataset Preparation

The audio samples were standardized to have a sampling frequency of 16kHz and to be in the mono format because being processed by the CNN feature extraction pipeline. Each sample was assinged to one of the three target classes.

E.4. Dataset Distribution

The final dataset contains 12,466 verified one-second audio clips. The dataset is divided into training, validation, and testing subsets as shown in Table E-4.

Table E-4. Dataset distribution

Subset	Samples	Percentage	Purpose
Training	8,750	70.18%	Model training
Validation	1,840	14.76%	Model tuning and validation
Testing	1,876	15.06%	Final model evaluation
Total	**12,466**	**100%**	**Complete dataset**

E.5. Feature Extraction Configuration

One second audio sample is converted into a log mel representation onto which a 512 point FFT with a hop length of 320 samples were used to generate the time-frequency representation. The obtained spectrum was then projected onto a 64 Mel-frequency bands which was then followed by logarithmic transformation and normalization.

This pipeline process remains the same during both model development and the embedded inference.

E.6. Model Development Summary

The prepared dataset is used in the teacher-student learning process as well as the final DS-CNN classifier. During the development process, the teacher model is used to provide the acoustic classification knowledge, the compact DS-CNN is optimized for smooth deployment on the ESP32-S3. The final model is tested using the unused test dataset which is thereafter quantized to INT8 for deployment in the embedded device.

E.7. Dataset Split Summary

Table E-7. Summary of the dataset and model-development pipeline

Parameter	Description
Total Audio Clips	12,466 verified one-second clips
Training Samples	8,750
Validation Samples	1,840
Testing Samples	1,876
Sampling Frequency	16 kHz
Audio Channel	Mono
Input Representation	Log-Mel spectrogram
Mel Bands	64
Target Classes	3
Final Embedded Model	Quantized DS-CNN
Deployment Platform	ESP32-S3

BIBLIOGRAPHY

[1] W. Ellermeier and K. Zimmer, "Individual differences in susceptibility to the irrelevant speech effect," *The Journal of the Acoustical Society of America*, vol. 102, pp. 2685–2691, Nov 1997.

[2] M. Haka, A. Haapakangas, J. Keranen, J. Hakala, E. Keskinen, and V. Hongisto, "Performance effects and subjective disturbance of speech in acoustically different office types – a laboratory experiment," *Indoor Air*, vol. 19, no. 6, pp. 454–467, 2009.

[3] J. Radun, J. Ljung, and V. Hongisto, "The effects of irrelevant speech on physiological stress, cognitive performance, and subjective experience – focus on heart rate variability," *International Journal of Psychophysiology*, vol. 200, p. 112352, Apr 2024.

[4] E. Lombard, "Le signe de l'elevation de la voix," *Annales des Maladies de l'Oreille, du Larynx, du Nez et du Pharynx*, vol. 37, pp. 101–119, 1911.

[5] P. Bottalico, I. I. Passione, S. Graetzer, and E. J. Hunter, "Evaluation of the starting point of the lombard effect," *Acta Acustica united with Acustica*, vol. 103, no. 1, pp. 169–172, 2017.

[6] P. Bottalico, R. N. Piper, and B. Legner, "Lombard effect, intelligibility, ambient noise, and willingness to spend time and money in a restaurant amongst older adults," *Scientific Reports*, vol. 12, no. 1, p. 6549, 2022.

[7] "ISO 22955:2021, acoustics – acoustic quality of open office spaces." International Organization for Standardization, Geneva, Switzerland, 2021.

[8] "ANSI/ASA S12.60-2010/Part 1, american national standard acoustical performance criteria, design requirements, and guidelines for schools, part 1: Permanent schools." Acoustical Society of America, Melville, NY, USA, 2010.

[9] K. S. Pearsons, R. L. Bennett, and S. Fidell, "Speech levels in various environments," Tech. Rep. EPA-600/1-77-025, U.S. Environmental Protection Agency, May 1977.

[10] A. N. Rimell, N. J. Mansfield, and G. S. Paddan, "Design of digital filters for frequency weightings required for risk assessments of workers exposed to noise," *Industrial Health*, vol. 53, no. 1, pp. 21–27, 2015.

[11] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, pp. 297–301, Apr 1965.

[12] S. Hershey *et al.*, "CNN architectures for large-scale audio classification," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, (New Orleans, LA, USA), pp. 131–135, 2017.

[13] J. Ramirez, J. M. Gorriz, and J. C. Segura, "Voice activity detection: Fundamentals and speech recognition system robustness," in *Robust Speech Recognition and Understanding* (M. Grimm and K. Kroschel, eds.), ch. 1, pp. 1–22, Vienna, Austria: I-Tech Education and Publishing, 2007.

[14] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O'Reilly Media, 1st ed., 2019.

[15] G. Cerutti, R. Prasad, A. Brutti, and E. Farella, "Compact recurrent neural networks for acoustic event detection on low-energy low-complexity platforms," *IEEE Journal of Selected Topics in Signal Processing*, vol. 14, pp. 786–797, May 2020.

[16] M. Hammad, A. M. Al-Masri, M. Al-Hammadi, and B. S. Al-Radaideh, "An unsupervised TinyML approach to detection of urban noise anomalies," *Internet of Things*, vol. 22, p. 100723, 2023.

[17] G. Marques and R. Pitarma, "A real-time noise monitoring system based on IoT for enhanced acoustic comfort and occupational health," *IEEE Access*, vol. 8, pp. 13974–13985, 2020.

[18] M. F. Usmani, "MQTT protocol for the IoT – review paper," *International Journal of Computer Applications*, vol. 183, pp. 17–22, Jun 2021.

[19] N. Radwan and J. Alves-Foss, "MQTT in focus: Understanding the protocol and its recent advancements," 2024.

[20] S. S. Gill *et al.*, "Edge AI: A taxonomy, systematic review and future directions," *Cluster Computing*, vol. 28, no. 1, p. 18, 2025.

[21] Espressif Systems, *ESP-DL: Deep Learning Library for Espressif SoCs*, 2023. [Online]. Available: `https://github.com/espressif/esp-dl`.